

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI



SMART BOARDING HOUSE MANAGEMENT SYSTEM

BACHELOR THESIS REPORT

RENTAL SAAS PLATFORM

Student

Đỗ Duy Toàn (22BI13420)

Supervisor: Supervisor Name

Hanoi, 2026

ABSTRACT

This project is about building a website called Smart Boarding House Management System. It is a SaaS platform that helps landlords and tenants manage their rental activities together. The system solves common problems in boarding houses, such as different room information, writing contracts by hand, delay in calculating monthly bills, lack of communication between landlords and tenants, and difficulties in tracking room issues or tenant rules violations.

To build this website, NestJS is used for the backend and Next.js for the frontend. PostgreSQL with Prisma is used to store main information like rooms and contracts. MongoDB is also used to keep activity logs, and Socket.IO to send real-time notifications to users. The platform has private zones for landlords and tenants. It includes many features: managing rooms and tenants, keeping track of contracts, generating monthly invoices automatically, checking payment status, exporting PDF bills with VietQR payment codes, reporting room problems, tracking tenant violations, and using a smart AI assistant to chat with tenants or write notifications.

The result of the project is a helpful system that saves time for landlords. It also gives tenants an easy way to check their contracts, bills, payments, and to report problems. This project is a great way to learn clean coding architecture, modular frontend design, user access security, automated tasks, and how to connect AI helper to a real business system.

TABLE OF CONTENTS

ABSTRACT	i
TABLE OF CONTENTS	iv
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	1
1 INTRODUCTION	2
1.1 Context and Motivation	2
1.2 Project Overview	2
1.3 Report Structure	3
2 OBJECTIVES	4
2.1 Desired Features	4
2.1.1 Public Rental Interface	4
2.1.2 Landlord Administration Dashboard	4
2.1.3 Tenant Portal	4
2.2 Project Scope	5
2.3 Engineering Goals	5
2.4 Expected Outcomes	5
3 REQUIREMENTS ANALYSIS	6
3.1 Overall System Requirements	6
3.1.1 Public Visitor Requirements	6
3.1.2 Landlord Requirements	6
3.1.3 Tenant Requirements	6
3.2 Use Cases	7
3.2.1 Landlord Use Case	7
3.2.2 Tenant Use Case	8

3.3	Use Case Specifications	9
3.3.1	Tenant Portal Use Cases	10
3.3.2	Landlord Dashboard Use Cases	15
3.4	Data Requirements	22
4	METHODOLOGY	24
4.1	Tools and Techniques	24
4.1.1	Backend Services	24
4.1.2	Frontend Portal	24
4.1.3	Supporting Services and Integrations	25
4.1.4	Infrastructure and Deployment	25
4.2	System Architecture	25
4.2.1	Overall System Structure	25
4.2.2	Layered Design	26
4.3	Database Design	27
4.3.1	Entity-Relationship Diagram	27
4.3.2	Core Database Tables	28
4.3.3	Multi-Tenant Data Privacy	29
4.4	Authentication and Authorization	29
4.4.1	User Authentication with JWT	29
4.4.2	API Protection with Auth Guards	30
4.4.3	Multi-Tenant Authorization Filter	30
4.5	Automation and Business Logic	31
4.5.1	Rental Contract Creation	31
4.5.2	Invoice Payment and Verification	33
4.5.3	Tenant Incident Handling	35
4.6	AI Integration	37
4.6.1	AI Assistant with Google Gemini	37
5	RESULTS AND DISCUSSION	39
5.1	Landlord Administration Dashboard and Room Management	39
5.2	Public Room Listing and RentBot AI Chatbot	40
5.3	Tenant Portal and Personal AI Assistant	41
5.4	Invoice Payment with VietQR Integration	43
5.5	Difficulties and Limitations	44
5.6	Discussion	45
6	CONCLUSION AND FUTURE WORK	47
6.1	Conclusion	47
6.2	Future Work	47

6.2.1	Production Payment Integration	47
6.2.2	Advanced Utility Billing	48
6.2.3	Stronger Multi-Tenant Security	48
6.2.4	Mobile Tenant Experience	48
6.2.5	Analytics and Forecasting	48
6.2.6	Document and Contract Management	48

LIST OF FIGURES

3.1	Use Case Diagram - Landlord Administration Dashboard	7
3.2	Use Case Diagram - Tenant Portal	9
3.3	Sequence Diagram - Tenant Log in	11
3.4	Sequence Diagram - View Contract	12
3.5	Sequence Diagram - View Invoices	13
3.6	Sequence Diagram - Report Incident	14
3.7	Sequence Diagram - Use AI Assistant	15
3.8	Sequence Diagram - Manage Rooms	16
3.9	Sequence Diagram - Manage Tenants	17
3.10	Sequence Diagram - Manage Contracts	19
3.11	Sequence Diagram - Manage Invoices	20
3.12	Sequence Diagram - Handle Incidents	21
3.13	Sequence Diagram - Track Violations	22
4.1	Overall System Architecture	26
4.2	Database Entity-Relationship Diagram	28
4.3	Authentication and Authorization Guard Flowchart	30
4.4	Activity Diagram - Contract Creation Process	32
4.5	Activity Diagram - Invoice Payment and Verification	34
4.6	Activity Diagram - Tenant Incident Handling	36
4.7	Contextual AI Prompt Construction Flowchart	38
5.1	Landlord Administration Dashboard — Room Management Overview	40
5.2	Public Room Listing Page with RentBot AI Chatbot	41
5.3	Tenant Portal Dashboard with Contract Summary and Personal AI Assistant	42
5.4	Invoice Payment Modal with VietQR Code and Bank Transfer Details	43

LIST OF TABLES

3.1	Summary of Landlord Use Cases	8
3.2	Summary of Tenant Use Cases	9
4.1	Structure of the JSON Web Token (JWT) Payload	30
4.2	Comparison of AI Assistant Modes and Data Contexts	37
5.1	Summary of Implemented System Features	44

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
VND	Vietnamese Dong
WebSocket	Web-based full-duplex communication protocol

Chapter 1

INTRODUCTION

1.1 Context and Motivation

In Vietnam, many landlords of boarding houses and small rental buildings still manage their business using simple methods. They often use paper notebooks, **Excel** files, social chat applications, or check bank transfers manually. Although these methods are easy to use at first, they become very hard to manage when the landlord has more rooms and tenants. Landlords have to spend a lot of time doing things like updating room availability, collecting tenant information, writing contracts, calculating bills every month, tracking payments, answering questions, and fixing broken facilities or handling rules violations.

Using different simple tools at the same time causes many problems. For example, room information can be different between what landlords post online and what they write in their private logs. It is also difficult to find old contracts and payment history. Tenants do not have a place to check their rental information, payment status, and reported issues by themselves. Also, when landlords calculate and write bills by hand, they can easily forget deadlines, make mistakes in calculations, or send payment notifications late.

Therefore, this project aims to build a website called **Smart Boarding House Management System** to put all these tasks together in one **SaaS** platform. This system has three main parts: a **public page** to show available rooms, a **landlord dashboard**, and a **tenant portal**. The system also has automatic and smart features like creating bills on schedule, creating **PDF** invoices with **VietQR** payment codes, sending notifications, and using **AI** to chat with users or generate announcements.

1.2 Project Overview

The system has two main user groups: **landlords** and **tenants**. The first group is **landlords**, who can manage rooms, tenants, contracts, invoices, reported incidents, consultation requests, and tenant violations. The second group is **tenants**, who can view their current contract, track

their invoices, report room incidents, receive notifications, and chat with an **AI** assistant that understands their rental details.

For the **backend**, **NestJS** is used to build a modular system. The system is organized into different modules like login, rooms, tenants, contracts, invoices, rent requests, incidents, violations, PDF generation, upload, notification, and AI assistant. The **frontend** is built using **Next.js**. It is divided into public pages, login pages, a landlord admin dashboard, and a tenant portal.

1.3 Report Structure

This report is divided into six chapters:

- **Chapter 1 (Introduction):** Introduces the project, its background, reasons for development, and system overview.
- **Chapter 2 (Objectives):** Describes the objectives, functional goals, scope of work, and expected results.
- **Chapter 3 (Requirements Analysis):** Shows the requirements analysis, including user roles, system requirements, use cases, and database details.
- **Chapter 4 (Methodology):** Explains the methodology, such as the chosen technologies, application architecture, database model, and step-by-step implementation.
- **Chapter 5 (Results and Discussion):** Discusses the results, focusing on the implemented features, testing, and limitations.
- **Chapter 6 (Conclusion and Future Work):** Concludes the report and suggests some future improvements.

Chapter 2

OBJECTIVES

2.1 Desired Features

The main goal of this project is to build a website for managing rental houses. The website helps manage everything from showing vacant rooms to the public, adding tenants, writing contracts, creating monthly bills, tracking payments, reporting issues, and helping landlords and tenants communicate easily.

2.1.1 Public Rental Interface

The public page allows people who want to rent a room to look at available rooms and send requests for consultation. It only shows vacant rooms and gives details like the price, room status, and description. In addition, a smart **AI** helper can answer common questions from users by using the room data in the database.

2.1.2 Landlord Administration Dashboard

The **landlord dashboard** is the main control center of the website. It gives **landlords** tools to manage rooms, add new tenants, write rental contracts, create monthly invoices, update payment status, manage room issues, track violations, and read room requests. The dashboard also keeps data private. This means each landlord can only see their own rooms, tenants, contracts, and invoices, and cannot see information from other landlords.

2.1.3 Tenant Portal

The **tenant portal** is a personal page for renters. Tenants can log in to check their contracts, see past bills, report issues in their rooms, read violation notices, mark invoices as paid, and get support from an **AI** assistant. This makes the rental process transparent and helps landlords and tenants save time because they do not have to ask the same questions repeatedly.

2.2 Project Scope

This project is designed for small and medium boarding houses, not for large real-estate companies. The main features in this project include user login with different roles for **landlords** and **tenants**, and keeping data private for each landlord. It also allows landlords to manage rooms and upload images, manage tenant accounts, and track contract status. In addition, the system can create monthly invoices automatically, export **PDF** bills, and show **VietQR** codes based on the landlord's bank account. Landlords can also track room incidents or tenant violations. Finally, the system sends real-time notifications to users and uses an **AI** assistant to support tenants and help landlords write notices.

2.3 Engineering Goals

Apart from building the application features, this project aims to achieve several engineering goals. First, the backend uses **Clean Architecture** rules and is built with **NestJS**. Second, the frontend is built with **Next.js** and has private pages. Third, **PostgreSQL** database with **Prisma** is used to keep all data consistent. The system also uses **JWT** and security guards to protect the API routes. Moreover, scheduled cron jobs are used to create monthly invoices and check expired contracts automatically. Finally, the system connects with third-party services like **Cloudinary** for image upload, **Google Generative AI** for the assistant, and **Socket.IO** for real-time messages.

2.4 Expected Outcomes

The expected result is a working prototype of a rental **SaaS** platform. This prototype can be a good foundation to build a complete boarding house management system in the future. For **landlords**, the system will reduce manual paperwork and make it easier to see how their business is running. For **tenants**, the system will help them check their bills, payment status, and get support easily and quickly.

Chapter 3

REQUIREMENTS ANALYSIS

3.1 Overall System Requirements

The system is designed around three access contexts: **public visitors**, **landlords**, and **tenants**. Each context has different permissions and workflows.

3.1.1 Public Visitor Requirements

Public visitors do not need to log in. They must be able to browse vacant rooms, view room information, submit consultation requests, and ask basic questions through the public **AI** assistant.

3.1.2 Landlord Requirements

Landlords must authenticate before accessing the **administration dashboard**. For landlords, the system requires several core features. First, landlords need room management to create, update, delete, list, and upload images of rooms. Rooms can have status like **vacant**, **occupied**, or **maintenance**. Second, landlords need tenant management to view tenants, approve or reject tenant registration requests, and connect tenants with rooms. Third, contract management is necessary to create rental contracts, record rental price and deposit, and update contract status. Fourth, invoice management allows landlords to create bills, review payment history, check payment status, and export invoices to **PDF** files. Fifth, operational support helps landlords review room incidents, track tenant violations, and receive real-time notifications. Finally, landlords can use **AI** assistance to generate announcements and notices for tenants quickly.

3.1.3 Tenant Requirements

Tenants must authenticate before accessing private rental information. For tenants, the system requires access to private rental data after they log in. Tenants must be able to view their active and past contracts, check invoices, and see if they have paid. They also need to mark invoices

as paid so that the landlord can check and confirm their payment. If there is a problem in the room, tenants must be able to report incidents. Additionally, they can see violation notices and ask questions to the **AI** assistant about their contract or bills.

3.2 Use Cases

3.2.1 Landlord Use Case

The landlord actor manages the boarding house operation through the administration dashboard. The main workflow groups are room management, tenant management, contract and invoice processing, rent request handling, and operational support.

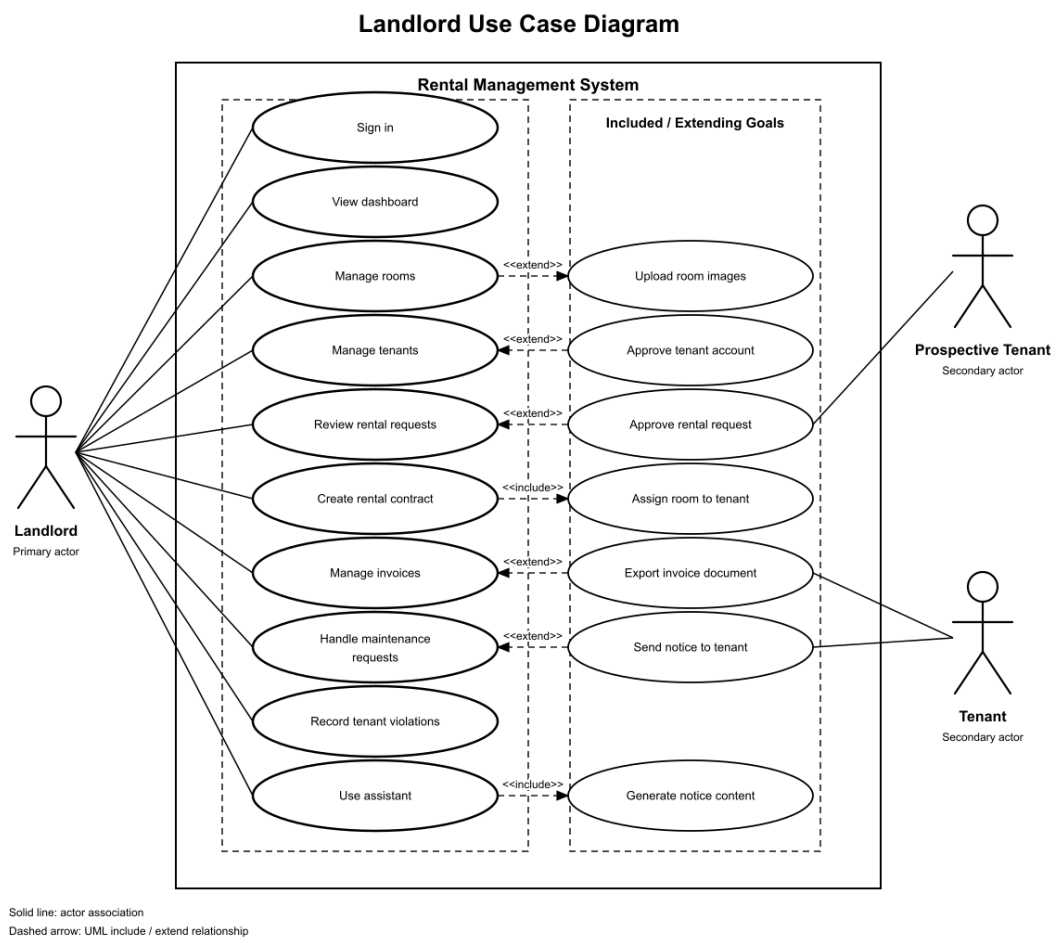


Figure 3.1: Use Case Diagram - Landlord Administration Dashboard

ID	Use Case Name	Description
UC-LAN-01	Manage rooms	Create, edit, delete, and update room availability.
UC-LAN-02	Manage tenants	Review tenant accounts and tenant information.
UC-LAN-03	Manage contracts	Create and update rental contracts.
UC-LAN-04	Manage invoices	Create invoices, update payment status, and export invoice PDFs.
UC-LAN-05	Handle incidents	Review and update tenant-reported maintenance requests.
UC-LAN-06	Track violations	Record and resolve tenant violations.
UC-LAN-07	Use AI assistance	Generate formal announcements and notices for tenants using the AI service.

Table 3.1: Summary of Landlord Use Cases

3.2.2 Tenant Use Case

The tenant actor uses the portal to monitor their rental relationship with the landlord, especially contract and payment information.

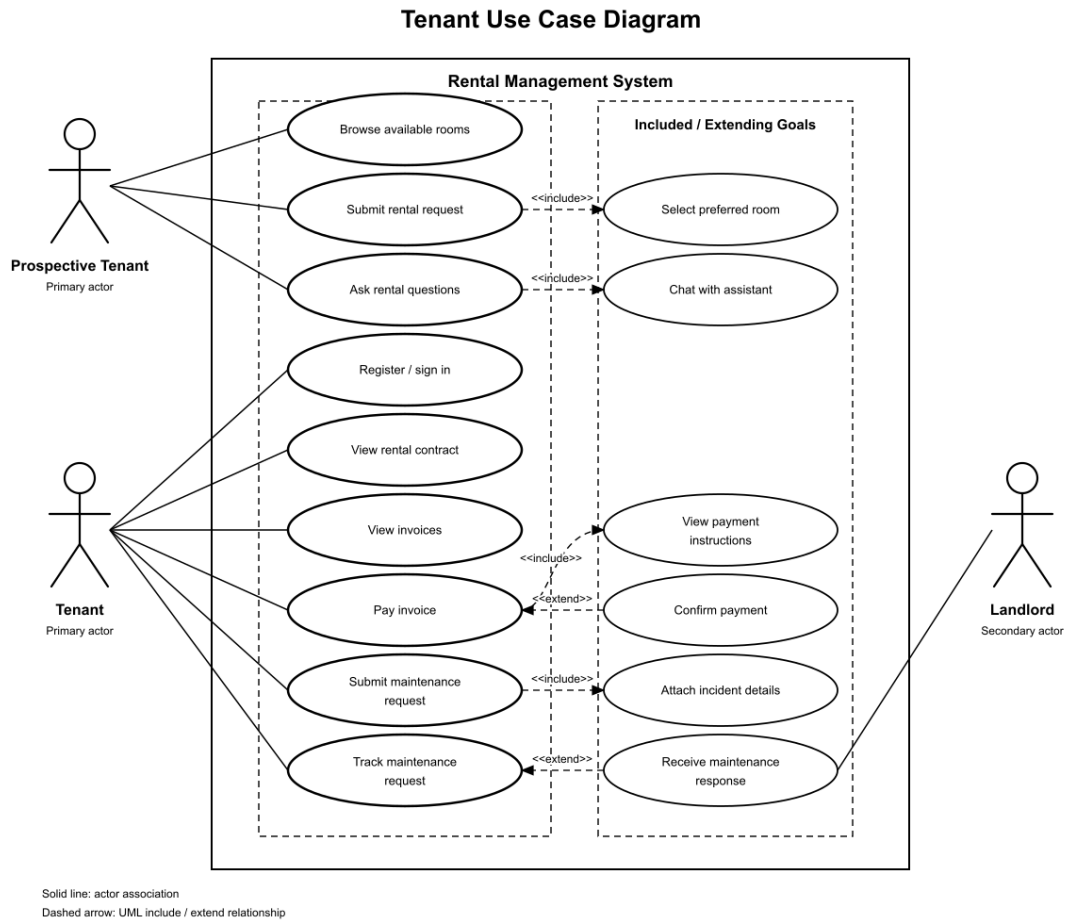


Figure 3.2: Use Case Diagram - Tenant Portal

ID	Use Case Name	Description
UC-TEN-01	Log in	Authenticate and access the tenant portal.
UC-TEN-02	View contract	Review active rental contract information.
UC-TEN-03	View invoices	Track invoices, due dates, and payment status.
UC-TEN-04	Report incident	Submit a repair or support request for the rented room.
UC-TEN-05	Use AI assistant	Ask contextual questions based on contract and invoice data.

Table 3.2: Summary of Tenant Use Cases

3.3 Use Case Specifications

This section provides a detailed description of the execution operations for specific Use Cases, outlining the interaction steps between the user and the system components.

3.3.1 Tenant Portal Use Cases

Tenant Log in

Goal: To allow tenants and landlords to log in to their account.

Primary Actor: Tenant or Landlord

Pre-conditions: The user has registered an account with a valid email and password, and has a device connected to the internet.

Main Success Scenario:

1. The user enters their email and password into the input fields.
2. The user clicks the "Log In" button.
3. The frontend validates the input format.
4. The system sends a request to the backend auth service.
5. The backend validates the credentials against the database.
6. The system generates a JWT token and returns it to the user.
7. The user is redirected to their dashboard page.

Alternative Flows: If the email or password is wrong, the system shows an incorrect password error and prompts the user to enter again. If there is a network error, the app displays a connection failure message.

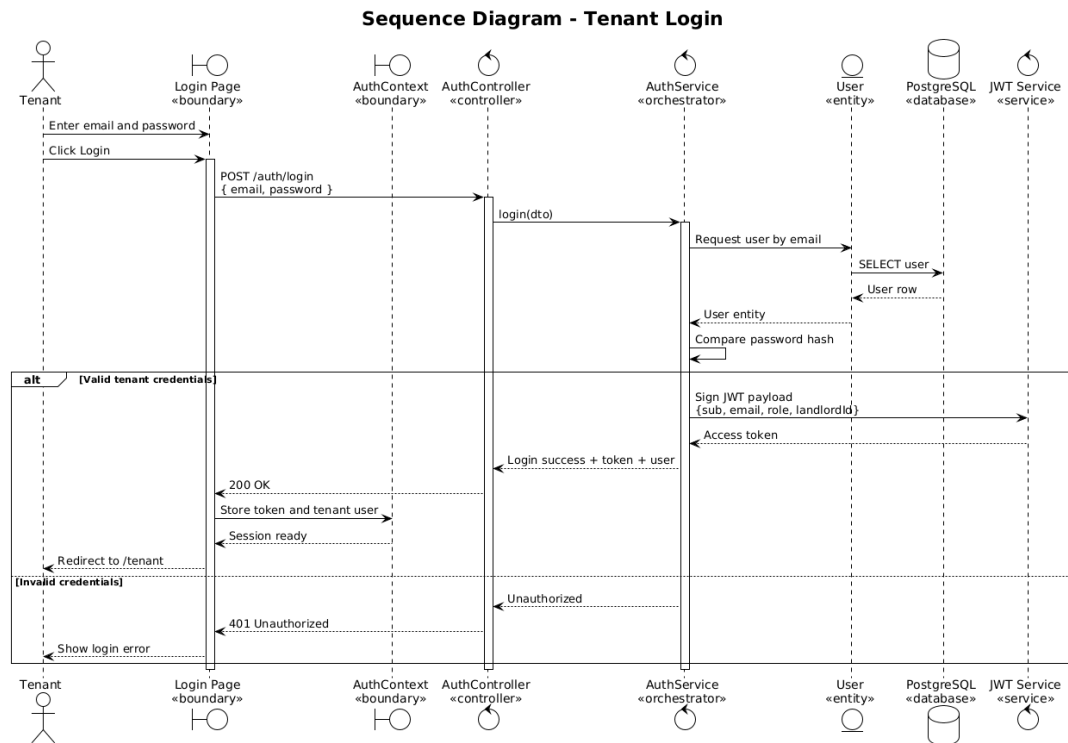


Figure 3.3: Sequence Diagram - Tenant Log in

View Contract

Goal: To allow the tenant to review their current rental contract details.

Primary Actor: Tenant

Pre-conditions: The tenant is logged in.

Main Success Scenario:

1. The tenant clicks the contract tab in the portal.
2. The frontend sends a request with the JWT token to the backend.
3. The backend fetches the contract details from the database.
4. The system displays the start date, end date, rental price, and deposit.

Alternative Flows: If the contract is not found or has expired, the system shows a message that there is no active contract.

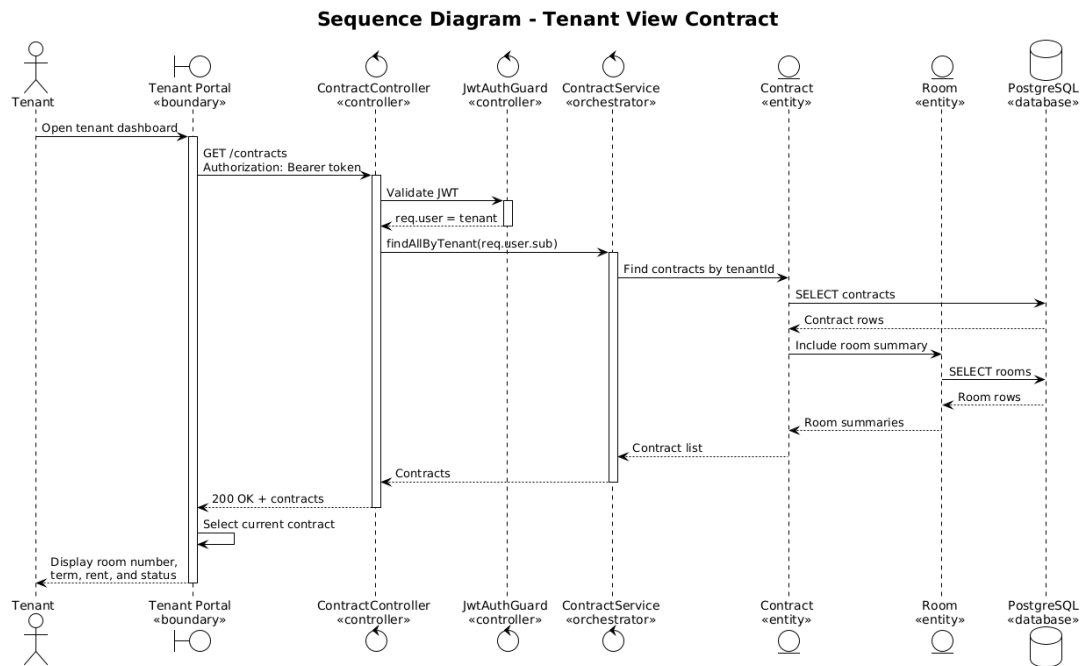


Figure 3.4: Sequence Diagram - View Contract

View Invoices

Goal: To check invoice history and payment status.

Primary Actor: Tenant

Pre-conditions: The tenant is logged in.

Main Success Scenario:

1. The tenant clicks on the billing section.
2. The frontend sends a request with the JWT token to the backend.
3. The backend queries the database for all invoices related to the tenant's contract.
4. The system displays the list of invoices with their status like unpaid, paid, or overdue.

Alternative Flows: If the tenant has no invoices yet, the page shows an empty list.

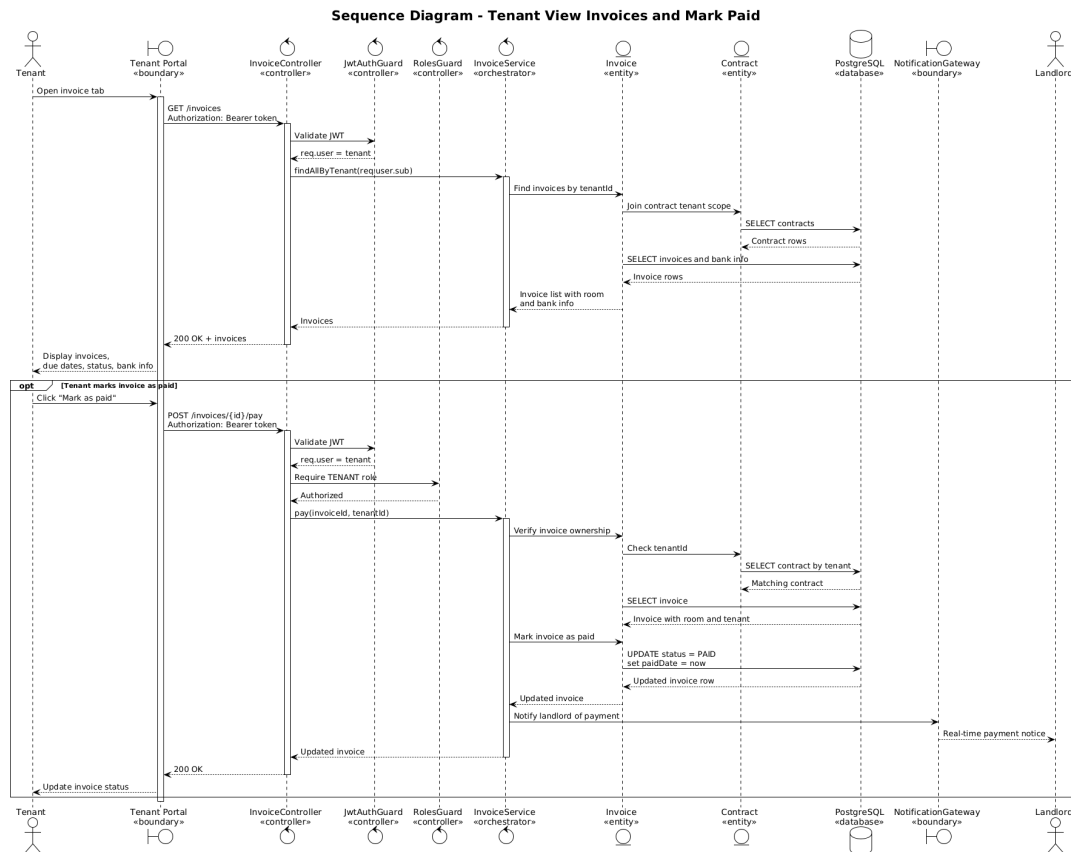


Figure 3.5: Sequence Diagram - View Invoices

Report Incident

Goal: To submit a room maintenance issue to the landlord.

Primary Actor: Tenant

Pre-conditions: The tenant is logged in.

Main Success Scenario:

1. The tenant opens the incident report form.
2. The tenant enters the title, description, and selects the priority level.
3. The tenant submits the form.
4. The system saves the incident in the database and updates the status to pending.
5. The system notifies the landlord in real time.

Alternative Flows: If the description is empty, the system prevents the submission and shows a validation error.

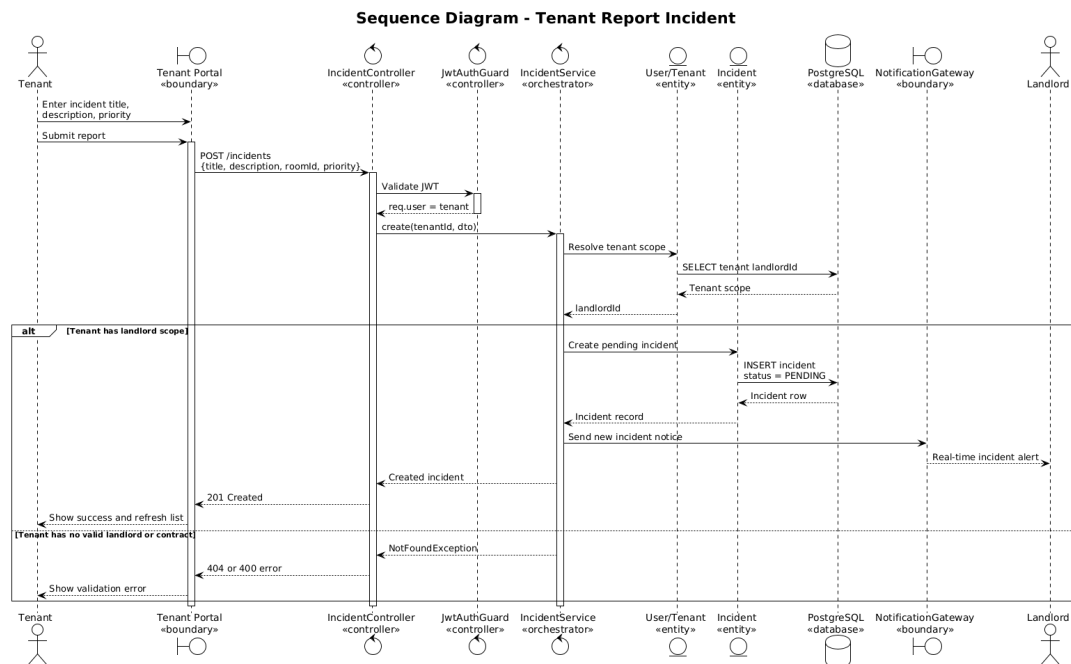


Figure 3.6: Sequence Diagram - Report Incident

Use AI Assistant

Goal: To chat with an AI assistant that knows the tenant's rental details.

Primary Actor: Tenant

Pre-conditions: The tenant is logged in.

Main Success Scenario:

1. The tenant opens the AI assistant chat box.
2. The tenant writes a question about their rental terms.
3. The backend fetches the tenant's contract and invoice data.
4. The backend sends the question and the fetched data to the Google Gemini model.
5. The model generates a response, and the system displays it to the tenant.

Alternative Flows: If the AI service is busy or not working, the system returns a message asking the user to try again later.

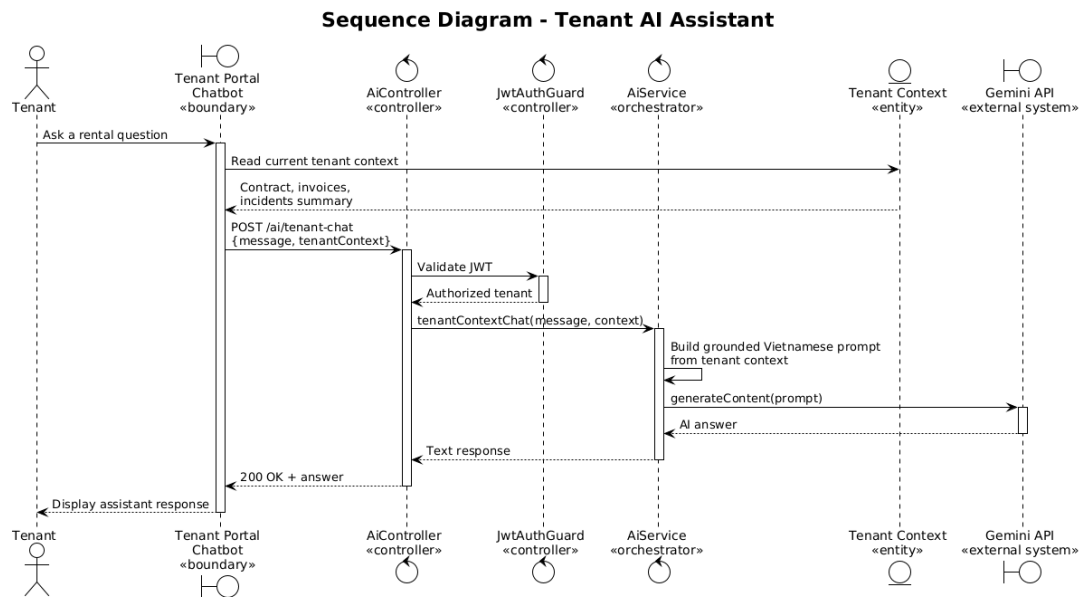


Figure 3.7: Sequence Diagram - Use AI Assistant

3.3.2 Landlord Dashboard Use Cases

Manage Rooms

Goal: To create, update, or delete room listings.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in.

Main Success Scenario:

1. The landlord clicks on the room management menu.
2. The landlord enters room details like room number, floor, price, description, and uploads images.
3. The landlord clicks "Save".
4. The backend saves the room information and uploads images to Cloudinary.
5. The updated room list is displayed.

Alternative Flows: If the room number already exists, the system returns a validation error.

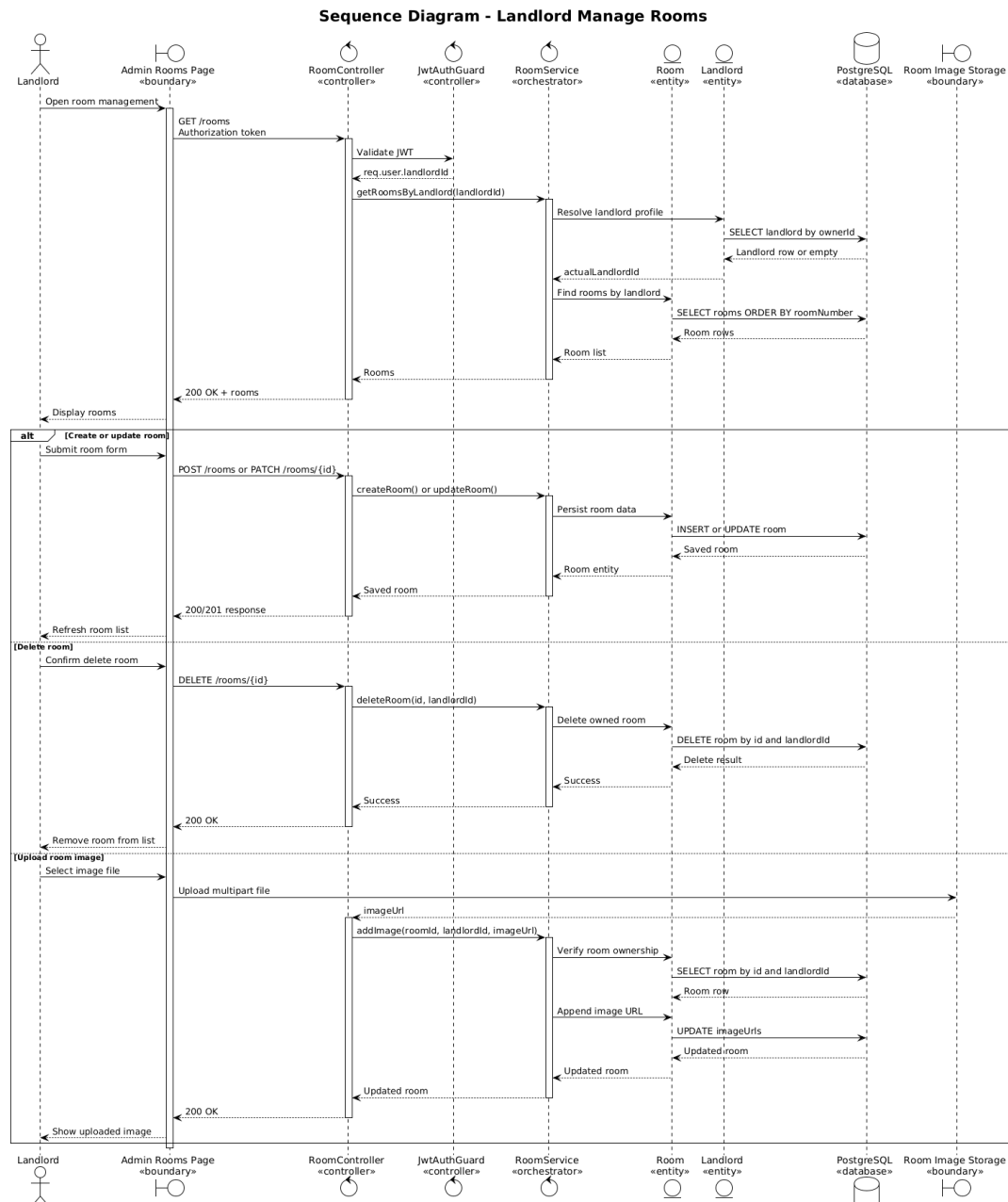


Figure 3.8: Sequence Diagram - Manage Rooms

Manage Tenants

Goal: To approve and manage tenant accounts.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in.

Main Success Scenario:

1. The landlord opens the tenant management page.
2. The landlord reviews the list of registered tenants and registration requests.

3. The landlord approves a tenant.
4. The system updates the tenant status in the database.

Alternative Flows: If the tenant's information is incomplete, the landlord can reject or suspend the account.

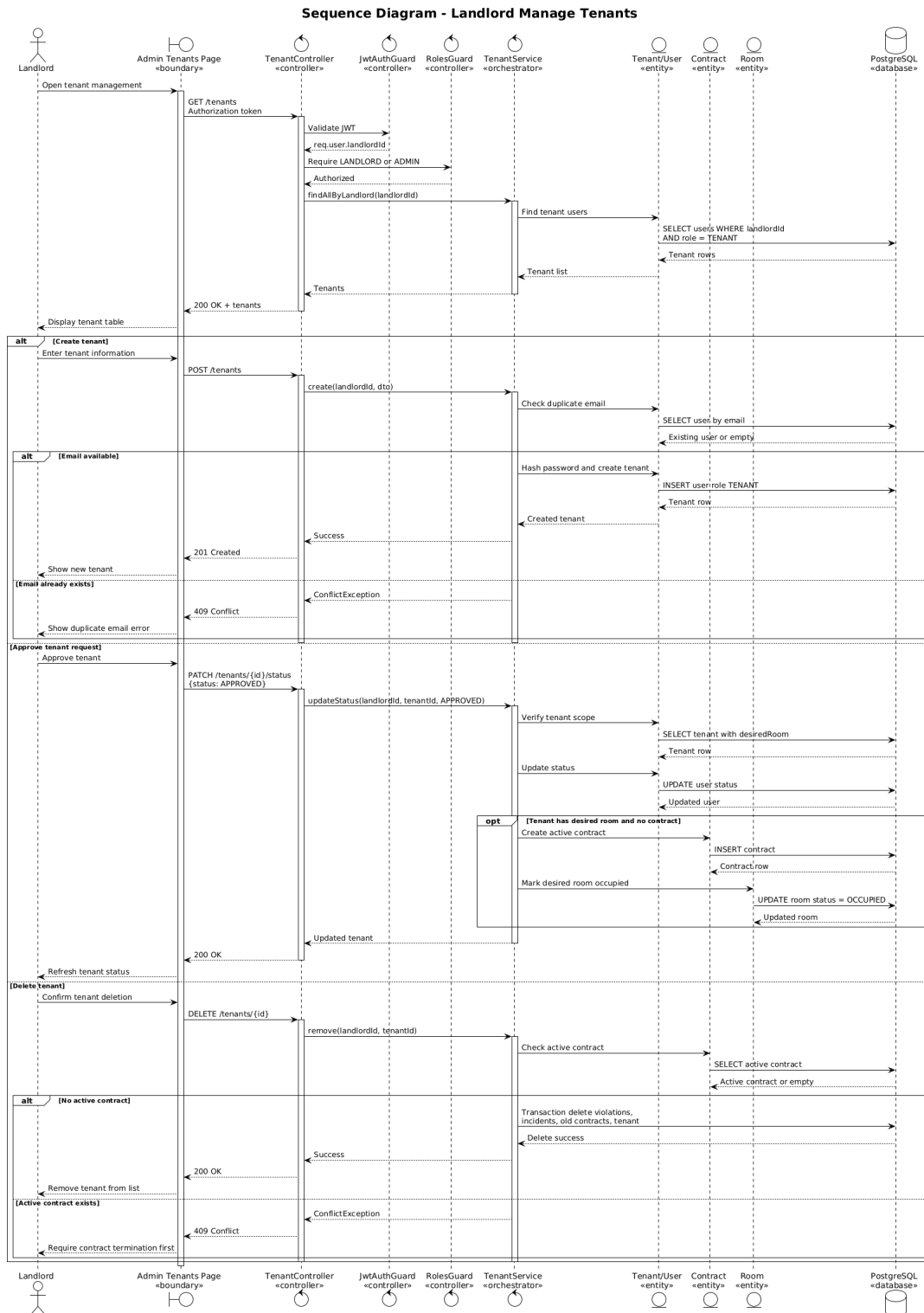


Figure 3.9: Sequence Diagram - Manage Tenants

Manage Contracts

Goal: To create a rental contract between a tenant and a room.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in. The room must be vacant, and the tenant must be approved.

Main Success Scenario:

1. The landlord enters contract details like tenant, room, dates, deposit, and rental price.
2. The landlord clicks "Save".
3. The backend validates tenant ownership and room status.
4. The system creates the contract inside a transaction.
5. The room status is updated to occupied.
6. The contract is shown in the dashboard.

Alternative Flows: If the room is occupied, the system rejects the request. If the tenant does not belong to this landlord, the system returns a validation error.

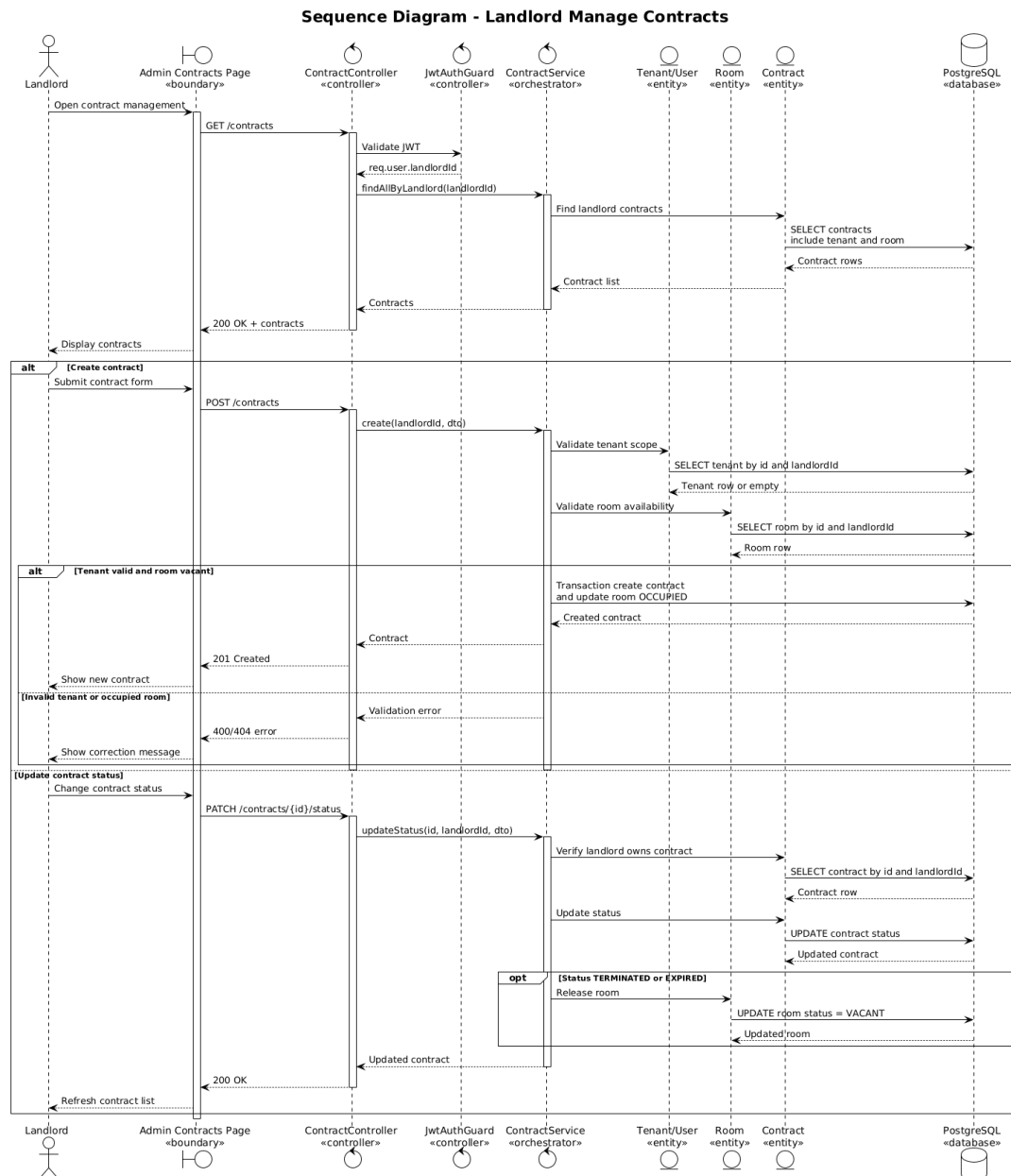


Figure 3.10: Sequence Diagram - Manage Contracts

Manage Invoices

Goal: To create invoices, check payments, and export PDF bills.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in, and an active contract exists.

Main Success Scenario:

1. The landlord goes to the billing section.
2. The landlord enters details like monthly price, electric usage, and water usage.
3. The landlord saves the invoice.

4. The system calculates the total amount, saves the invoice, and creates a PDF file.
5. The landlord can view the PDF invoice with a VietQR code.

Alternative Flows: If the invoice details contain invalid values, the system returns an error.

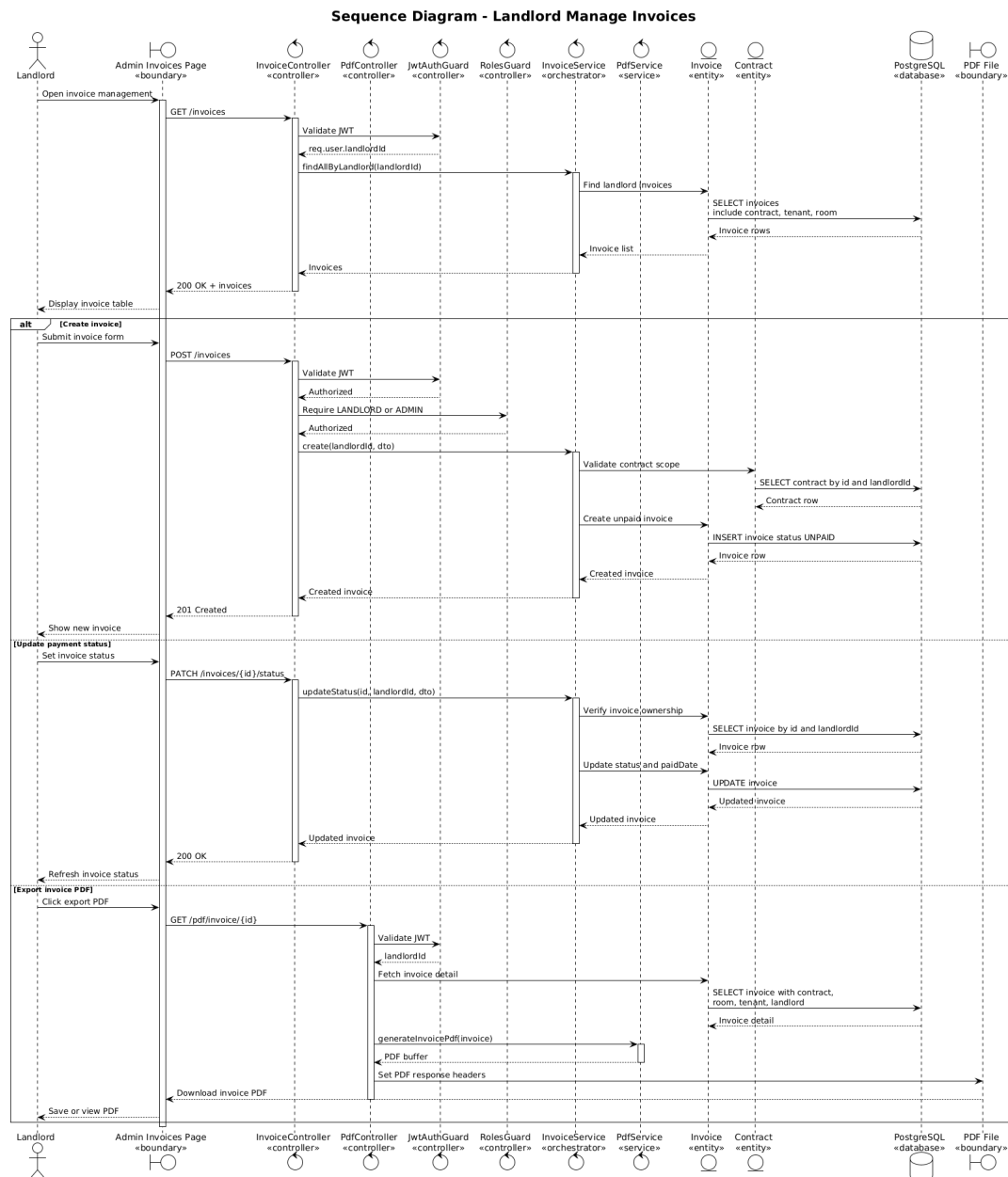


Figure 3.11: Sequence Diagram - Manage Invoices

Handle Incidents

Goal: To review and solve tenant-reported incidents.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in.

Main Success Scenario:

1. The landlord opens the incident management list.
2. The landlord reviews the pending issues reported by tenants.
3. The landlord updates the incident status to "In Progress" or "Resolved".
4. The system saves the status in the database and notifies the tenant in real time.

Alternative Flows: If the incident has already been resolved by another admin, the system prevents duplicate actions.

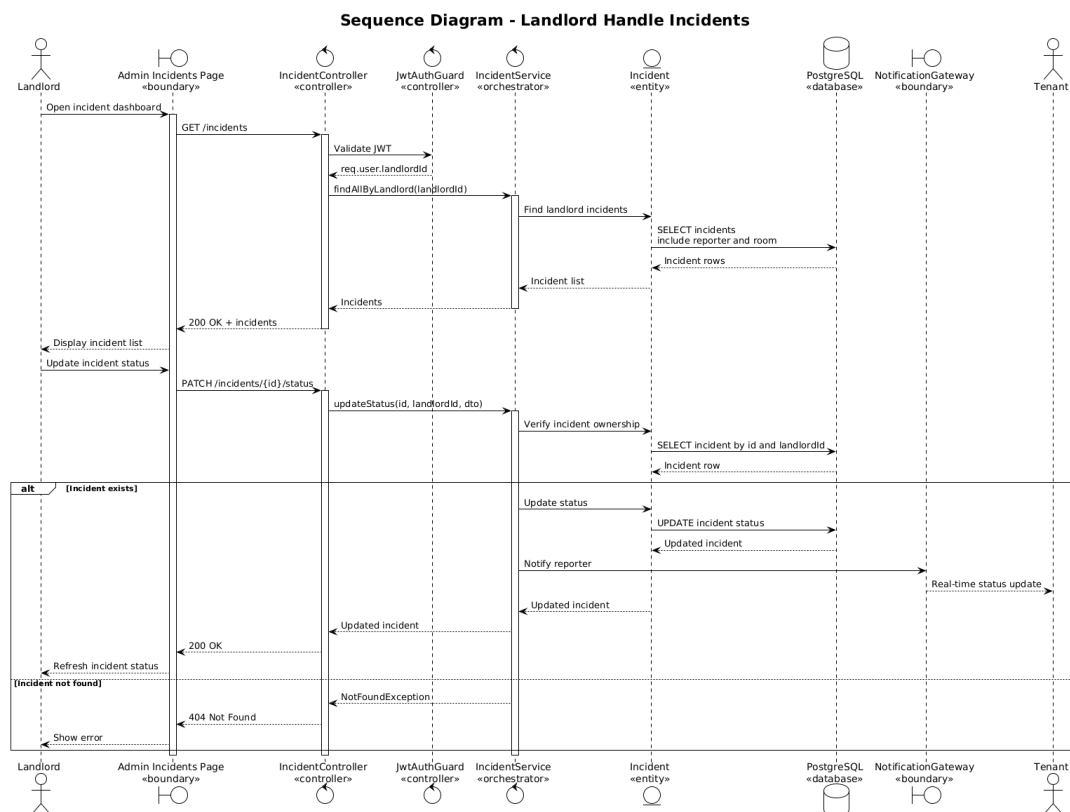


Figure 3.12: Sequence Diagram - Handle Incidents

Track Violations

Goal: To record rules violations for tenants.

Primary Actor: Landlord

Pre-conditions: The landlord is logged in.

Main Success Scenario:

1. The landlord opens the violations page.
2. The landlord enters a description of a rule violation for a tenant.
3. The landlord clicks "Add".

4. The system saves the violation record and makes it visible to the tenant.

Alternative Flows: If the target tenant does not exist or has left, the system returns an error.

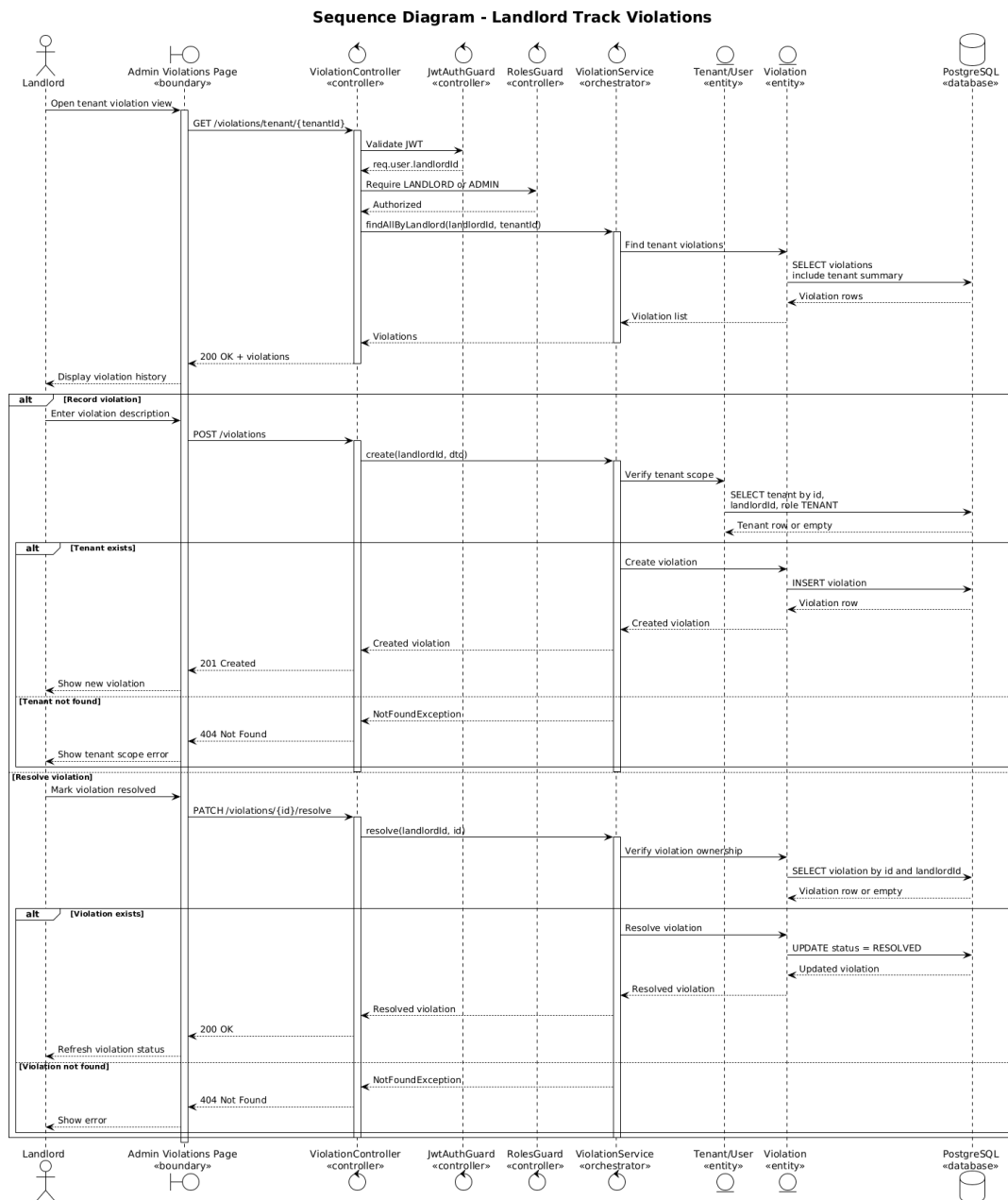


Figure 3.13: Sequence Diagram - Track Violations

3.4 Data Requirements

The core database model uses a relational structure. The main entities are users, landlords, rooms, contracts, invoices, rent requests, incidents, and violations. Each record is scoped by a landlord identification code. This keeps the data private for each landlord. Main status fields include room availability, contract status, invoice payment status, user registration status, and

incident resolution status.

Chapter 4

METHODOLOGY

This chapter presents the architecture and implementation methods used to build the *Smart Boarding House Management System*.

4.1 Tools and Techniques

Choosing the right tools is very important to build a reliable rental management platform. This section explains the technologies selected for the backend, frontend, supporting services, and system deployment, and the reasons for choosing them instead of other options.

4.1.1 Backend Services

For the backend, **NestJS** is chosen instead of simple frameworks like **Express**. **NestJS** provides a modular structure, dependency injection, and decorators, which makes it easy to organize the codebase. This organization is very helpful for implementing **Clean Architecture** and keeping the code neat as the system grows. **PostgreSQL** is used as the primary database because rental data like contracts and invoices must be secure and consistent. To connect the backend with **PostgreSQL**, **Prisma ORM** is used because it offers type-safe queries and makes database migrations easy. While **MongoDB** schemas are present in the codebase for potential logging development, the current active implementation relies entirely on **PostgreSQL** for data consistency.

4.1.2 Frontend Portal

For the frontend, **Next.js 14 App Router** is used instead of a standard **React** single-page application. **Next.js** offers server-side rendering, which is useful for the public landing page to load faster and have better search engine optimization. It also has a clean routing system that helps separate public pages, landlord admin pages, and the tenant portal. To manage the global application state, **Zustand** is chosen instead of **Redux**. **Zustand** is much lighter, has less boilerplate

code, and is easier to learn, while still providing great performance for storing user sessions and UI states. For API requests, **Axios** is used with custom interceptors to automatically attach landlord identification headers and authorization tokens to every request.

4.1.3 Supporting Services and Integrations

To add special features to the platform, several supporting libraries and services are integrated. **Socket.IO** is used instead of constant HTTP polling to send notifications. This allows landlords to receive instant alerts when a tenant reports an incident or sends a room request. To store room images, **Cloudinary** is chosen instead of saving files locally on the server, which helps save disk space and automatically optimizes images for faster loading. For AI features, the **Google Generative AI** library with the **Gemini** model is used because it is very fast, accurate, and cost-effective for generating landlord notices and chatting with tenants. Finally, the **VietQR** image API is used to generate payment QR codes and insert them into invoice **PDF** files, allowing tenants to pay easily using their banking apps.

4.1.4 Infrastructure and Deployment

For deployment and production hosting, the system uses a serverless cloud infrastructure. The database is hosted on **Neon**, which provides a serverless **PostgreSQL** database. This allows the system to store rental data securely with high availability. The **NestJS** backend is deployed as a Web Service on **Render**. **Render** automatically builds the **NestJS** application from the backend directory and connects it to the **Neon** database. The **Next.js** frontend is deployed on **Vercel** because **Vercel** is optimized for **Next.js** applications, offering fast loading speeds. The frontend communicates with the backend on **Render** using environment variables. Both **Render** and **Vercel** are connected to the **GitHub** repository. This setup enables automatic **CI/CD** deployment, meaning any new code changes pushed to the repository are automatically built and deployed to the production environment.

4.2 System Architecture

This section describes how the different parts of the Smart Boarding House Management System are organized and connect with each other. The system is designed using a clean, layered structure to separate the user interface, business logic, and database operations.

4.2.1 Overall System Structure

The system follows a **client-server** architecture. The frontend is built with **Next.js** and acts as the client side. The backend is built with **NestJS** and acts as the server side. As shown in Figure 4.1, the client and the server communicate using **HTTP REST APIs** for standard

data requests and **Socket.IO** for real-time notifications. The backend server connects directly to the **PostgreSQL** database to read and store information. It also communicates with external services like **Cloudinary** to upload images, **Google Gemini** to run **AI** features, and the **VietQR API** to get payment codes.

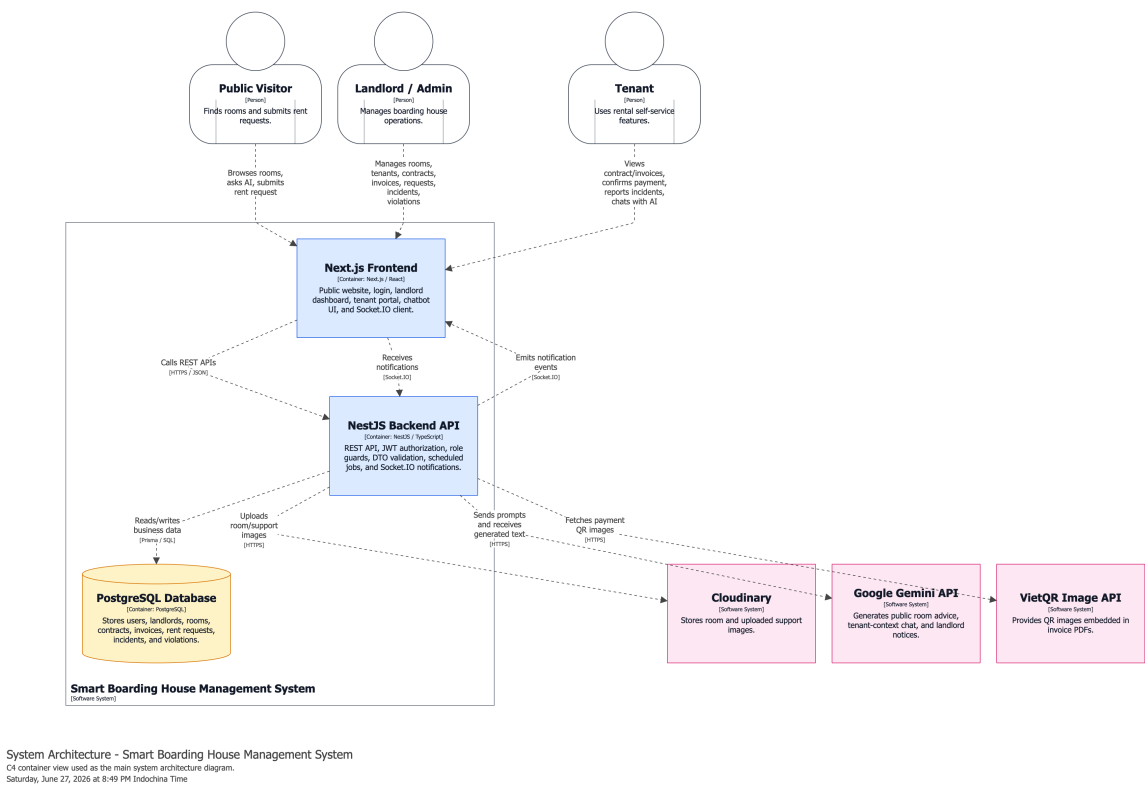


Figure 4.1: Overall System Architecture

4.2.2 Layered Design

To make the codebase easy to maintain, the **backend** and **frontend** are organized into three distinct layers: the **presentation layer**, the **application layer**, and the **infrastructure layer**.

The **presentation layer** is the top layer. In the **backend**, it contains HTTP controllers and data transfer objects to receive requests and send responses. In the **frontend**, it contains the visual pages and interactive components. The **presentation layer** does not process business rules; it only passes data to the next layer.

The **application layer** contains the core business logic of the system. For example, when a landlord creates a contract, the **application layer** checks if the room is vacant and updates the room status. It contains modules for authentication, room management, contract lifecycles, and invoice creation. This layer is completely independent of the database details or framework configurations.

The **infrastructure layer** is the bottom layer. It handles communication with external systems. In the **backend**, it includes database services to execute **PostgreSQL** queries using **Prisma**, and call external APIs like **Cloudinary** and **Google Generative AI**. This layered design ensures that changes in one part of the system, like upgrading the database, do not affect the rest of the application.

4.3 Database Design

The database is a very important part of the platform because it stores all information about rooms, tenants, contracts, and invoices. **PostgreSQL** is used as the database and **Prisma ORM** connects the backend server to it. This section explains the database structure, the relationships between different tables, and how the data is organized.

4.3.1 Entity-Relationship Diagram

To understand how the data is structured, an **Entity-Relationship Diagram (ERD)** is created. As shown in Figure 4.2, the database has eight main tables that represent the core parts of the boarding house system. These tables are **User**, **Landlord**, **Room**, **Contract**, **Invoice**, **RentRequest**, **Incident**, and **Violation**.

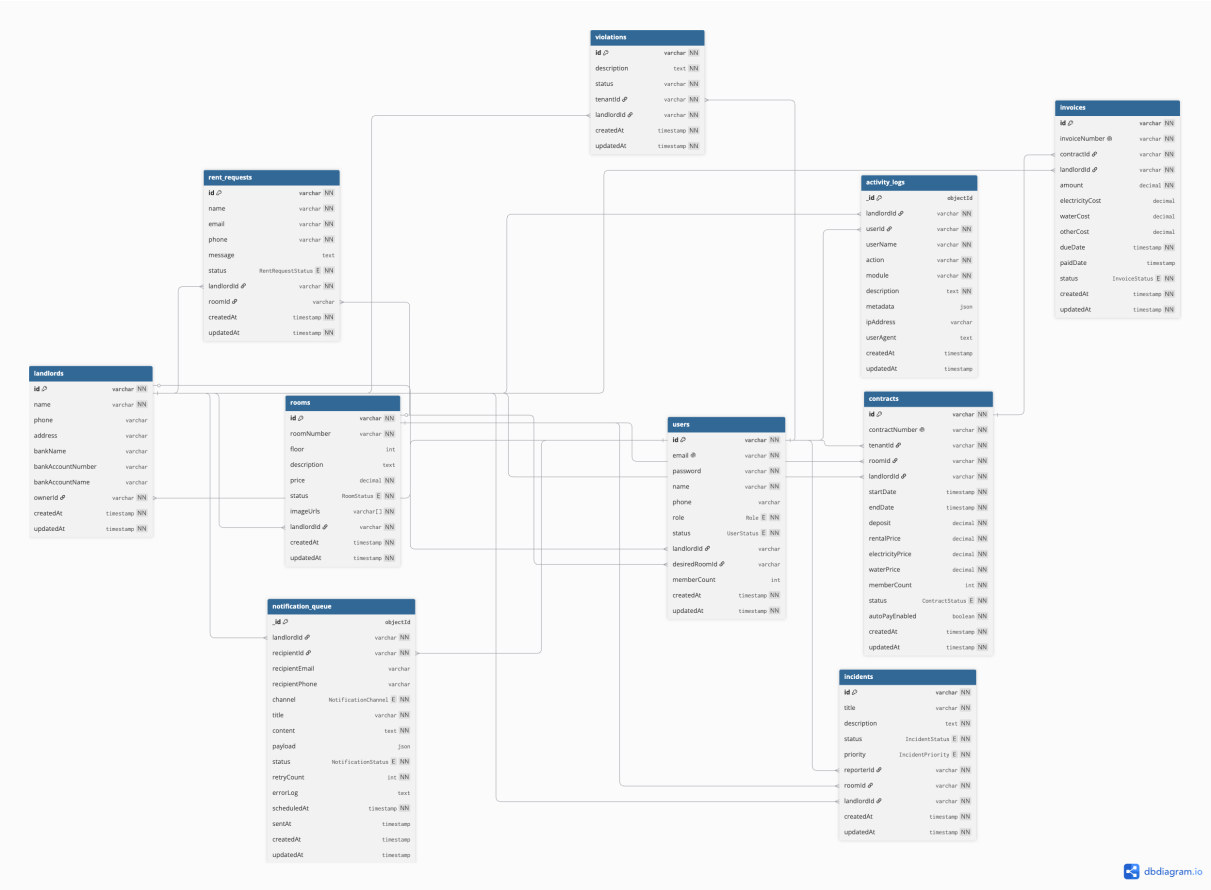


Figure 4.2: Database Entity-Relationship Diagram

4.3.2 Core Database Tables

The **User** table stores information about all accounts in the system. Each user has an email, password, name, phone, status, and role (which can be **Admin**, **Landlord**, or **Tenant**). If the user is a tenant, they can also have a **landlordId** to connect them to a specific landlord.

The **Landlord** table stores the business profile of property owners. It contains details like the landlord’s name, phone, address, and bank account information (bank name, account name, and account number). This bank information is used to generate **VietQR** codes on invoice **PDFs** automatically.

The **Room** table contains information about the rental rooms. Each room has a room number, floor, description, rental price, status (which can be **Vacant**, **Occupied**, or **Maintenance**), and a list of image URLs hosted on **Cloudinary**. Every room is connected to a specific landlord.

The **Contract** table represents the lease agreement. It connects a tenant user and a room together. It stores the start date, end date, deposit amount, monthly rental price, electricity price, water price, and status (which can be **Pending**, **Active**, **Expired**, or **Terminated**).

The **Invoice** table stores monthly billing records. It links to a specific contract and stores the total amount, electric cost, water cost, other costs, due date, payment date, and status (which

can be **Unpaid, Paid, Overdue, or Cancelled**).

The **RentRequest** table stores requests from public visitors who want to ask about rooms. The **Incident** table stores repair and maintenance requests submitted by tenants. The **Violation** table stores rules violations recorded by landlords.

4.3.3 Multi-Tenant Data Privacy

To keep the data of each landlord private, the system uses a multi-tenant design. Most tables in the database, including **Room, Contract, Invoice, RentRequest, Incident, and Violation**, contain a **landlordId** field. When a landlord logs in, the **backend** server automatically filters all queries using their **landlordId**. This ensures that a landlord can only see their own rooms, contracts, and invoices, and cannot access data belonging to other landlords.

4.4 Authentication and Authorization

Security is a critical part of the Smart Boarding House Management System. The platform must ensure that only registered users can access private pages, and that landlords can only see their own rental data. To implement this, the backend uses token-based authentication and role-based access control.

4.4.1 User Authentication with JWT

The system uses JSON Web Tokens (JWT) to authenticate users. When a user logs in by entering their email and password, the authentication service validates their credentials. If the information is correct, the server generates a signed JWT token and returns it to the client. This token contains a payload with the user's identification code, email address, role, and landlord identification code. For every subsequent request, the client must send this token in the HTTP Authorization header as a Bearer token. To validate these tokens, the backend uses a NestJS Passport strategy. This strategy extracts the token, verifies its signature using a secret key, and automatically attaches the decoded user data to the request object as the authenticated user. The structure of this JWT token payload is described in Table 4.1.

Claim Key	Data Type	Purpose and Description
sub	String (UUID)	The unique identifier of the authenticated User.
email	String	The email address used to identify the user account.
role	Enum	The system role of the user (e.g., ADMIN, LANDLORD, or TENANT).
landlordId	String (UUID)	The landlord identifier used to filter data access for multi-tenancy.

Table 4.1: Structure of the JSON Web Token (JWT) Payload

4.4.2 API Protection with Auth Guards

To protect individual API endpoints, the backend implements custom NestJS guards. The primary guard is the JWT Auth Guard, which intercepts incoming requests to protected routes. If the request does not contain a valid token, the guard blocks the request and throws an unauthorized exception. For routes that require specific roles, the system combines the JWT Auth Guard with a Roles Guard. The Roles Guard uses a custom Roles decorator and the NestJS Reflector to read the roles allowed for a route. It checks the role of the logged-in user inside the request object and compares it with the required roles. For example, if a route is decorated for landlords only, a tenant attempting to access it will be blocked and receive a forbidden exception. The flowchart in Figure 4.3 shows how requests are processed through these guards.

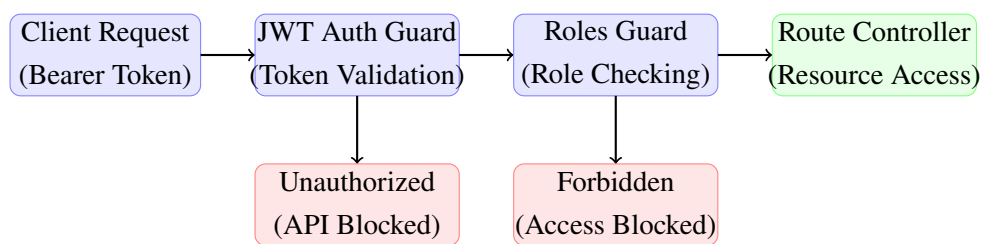


Figure 4.3: Authentication and Authorization Guard Flowchart

4.4.3 Multi-Tenant Authorization Filter

Apart from roles, the system enforces multi-tenant authorization using the landlord identification code. Because multiple landlords share the same database, the **backend** must prevent cross-tenant access. During authentication, the landlord's unique identification code is stored inside the **JWT** token. When a landlord requests room listings, contracts, or invoice records, the corresponding backend service reads this **landlordId** code directly from the request user object. The database query then filters the results based on this code. This technique guarantees

that landlords can never view or modify the records of other property owners, even if they try to bypass the frontend interface.

4.5 Automation and Business Logic

This section describes the technical implementation of key business processes in the **backend**. To ensure data consistency and provide a good user experience, the system automates several workflows using database transactions, real-time messaging, and scheduled background tasks. These automated tasks include contract expiration checks, which run daily to return rooms to vacant status when leases end, and startup consistency checks, which automatically fix any room marked occupied that lacks an active contract.

4.5.1 Rental Contract Creation

Creating a new rental contract requires updating multiple records at the same time. As shown in the activity diagram (Figure 4.4), when a landlord creates a contract, the system must verify room availability, save the contract details, and update the room status to occupied. To implement this safely, the **NestJS** backend uses a **Prisma** database transaction. This transaction guarantees that if any step fails (for example, if the tenant is invalid), the database rolls back to its previous state. This prevents bugs where a contract is created but the room remains vacant, or a room is marked occupied without an active contract.

Activity Diagram - Create Rental Contract

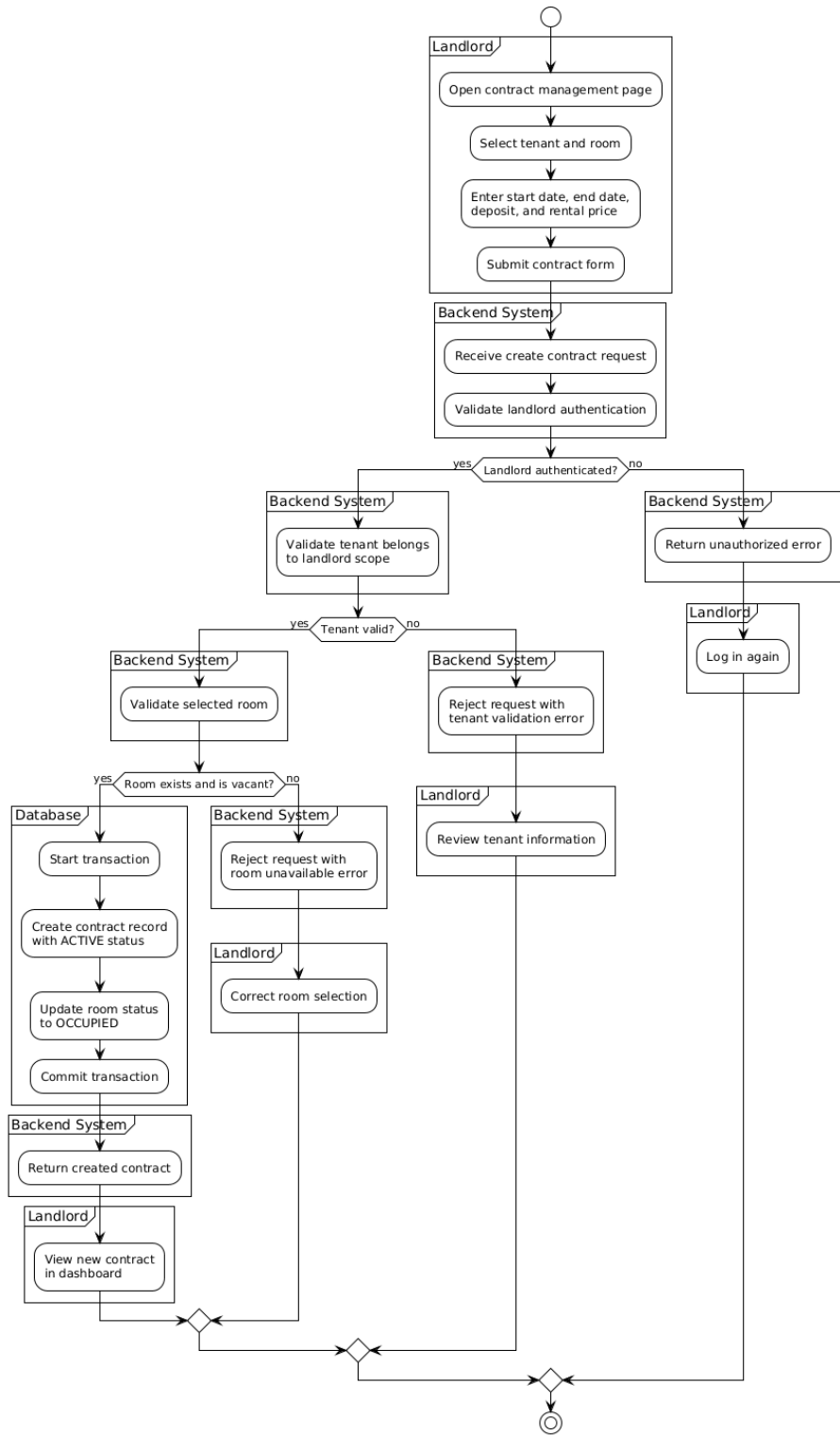


Figure 4.4: Activity Diagram - Contract Creation Process

4.5.2 Invoice Payment and Verification

The monthly billing cycle is managed through automated invoice generation and a structured payment verification flow. At the beginning of each month, a **NestJS** cron job runs automatically to create unpaid invoices for all active contracts. As shown in the activity diagram (Figure 4.5), once an invoice is created, the tenant can view it and scan the generated **VietQR** code to make a bank transfer. After transferring the money, the tenant marks the invoice as paid in the portal, which immediately changes its status to **Paid** and records the payment timestamp. The backend simultaneously broadcasts a real-time **Socket.IO** notification to the landlord's dashboard. The landlord can then manually verify the bank transfer and, if necessary, adjust the invoice status through the administration panel.

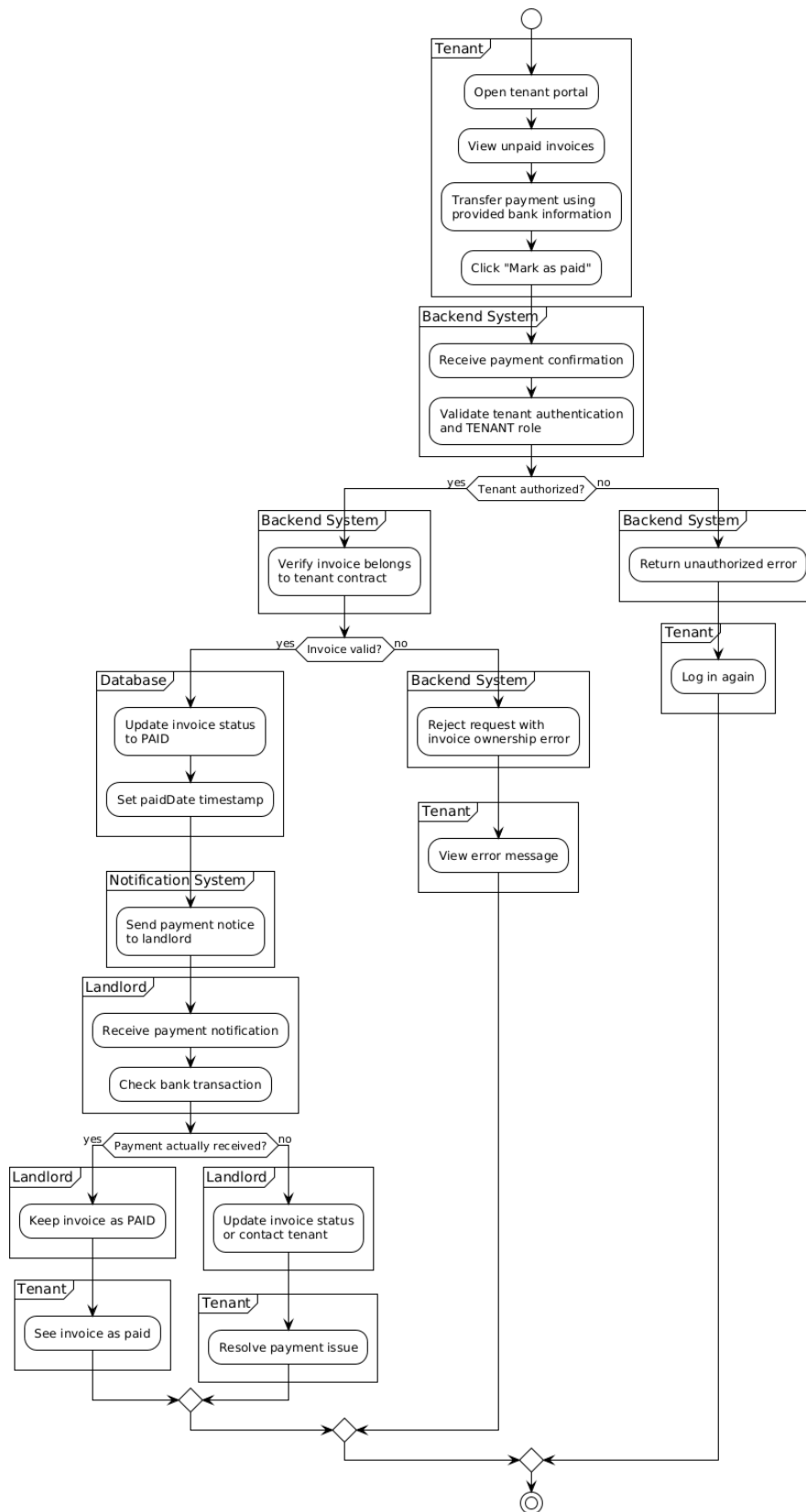
Activity Diagram - Invoice Payment and Verification

Figure 4.5: Activity Diagram - Invoice Payment and Verification

4.5.3 Tenant Incident Handling

Managing room repairs quickly requires close cooperation between tenants and landlords. As shown in the activity diagram (Figure 4.6), the process starts when a tenant submits a new incident report with a priority level. The **backend** saves the incident in the database with a pending status and immediately broadcasts a **WebSocket** event to the landlord's dashboard using **Socket.IO**. The landlord reviews the incident, assigns a technician, and updates the status to in-progress. Once the issue is fixed, the landlord marks it as resolved. The system updates the status in the database and sends a real-time message back to the tenant's portal to notify them.

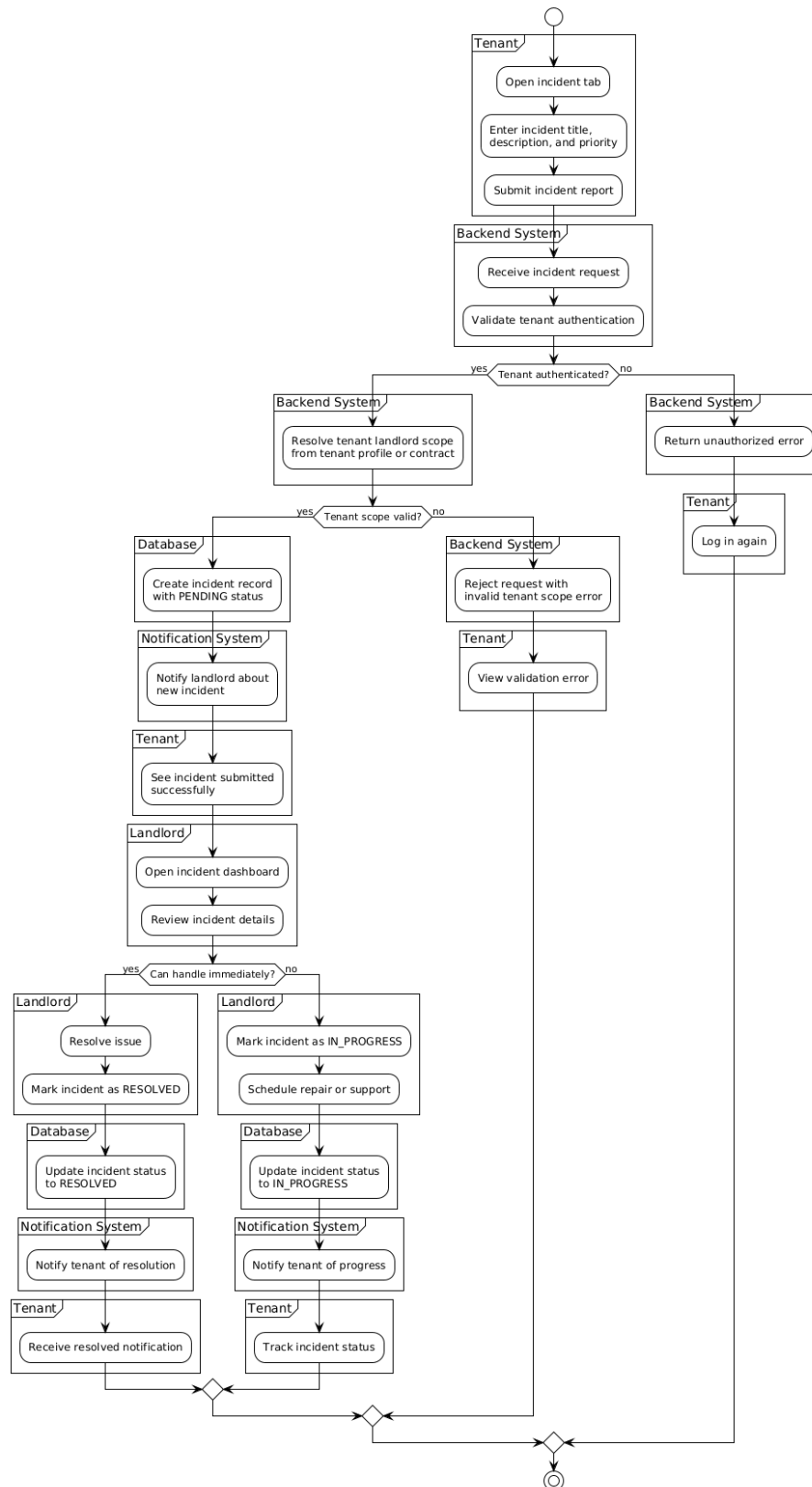
Activity Diagram - Tenant Incident Handling

Figure 4.6: Activity Diagram - Tenant Incident Handling

4.6 AI Integration

The platform integrates advanced artificial intelligence and optical character recognition to automate administrative workflows and improve communication. This section describes how the AI assistant and the OCR utility meter reader are built and how they process data.

4.6.1 AI Assistant with Google Gemini

The **AI** service is built using the official **Google Generative AI** library and the **Gemini 3.1 Flash Lite** model. This model was chosen because it provides fast responses and is highly accurate for text processing. The system implements three distinct assistant modes to help different users, as summarized in Table 4.2.

Assistant Mode	Database Context Injected	Core Capability
Notice Generator	None (Direct prompt input)	Generates polite, formal boarding house announcements in Vietnamese.
RentBot (Public)	Room number, rental price, floor, status, description.	Answers visitor questions about vacant rooms and directs them to register.
Tenant Assistant	Contract dates, monthly rent, deposits, unpaid invoices, violations, incidents.	Provides personalized rental Q&A, checks bills, and explains rules.

Table 4.2: Comparison of AI Assistant Modes and Data Contexts

The first mode is notice generation for landlords. Landlords often need to send announcements to all tenants, which can take time to write politely. The landlord can type a simple request in the dashboard, and the **AI** service automatically generates a formal, polite, and clear announcement in Vietnamese, ready to be sent.

The second mode is the public room consultation chatbot. When visitors browse the landing page, they can chat with an assistant named **RentBot**. To prevent the **AI** from making up incorrect information, the **backend** first queries the **PostgreSQL** database for actual room numbers, prices, and availability status. The system formats this room list as context and sends it along with the user’s question to the **Gemini** model. This ensures that the assistant only gives accurate answers about vacant rooms and instructs visitors how to apply.

The third mode is the contextual tenant portal assistant. Inside the tenant portal, renters can chat with a personal assistant. When a tenant opens the chat, the **backend** service queries

the database for their specific active contract, invoice history, reported incidents, and open rule violations. It formats this personal data into a detailed system context for the **Gemini** model. Because the **AI** has this context, the tenant can ask personalized questions like when their rent is due, how much they owe, or if their reported incident is resolved. The **AI** answers using the real-time data, providing a personalized self-service experience. The flow of how this personalized prompt is constructed is shown in Figure 4.7.

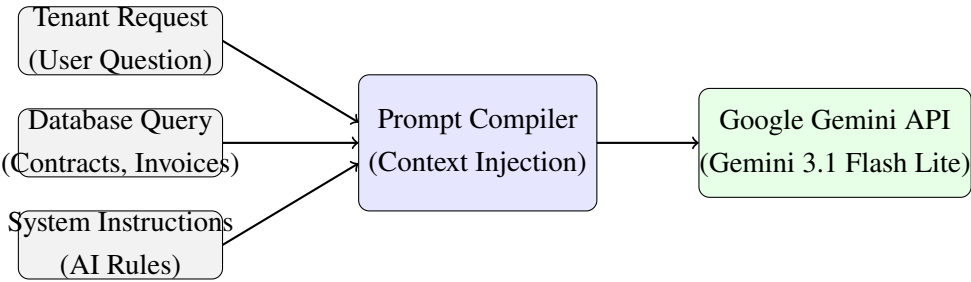


Figure 4.7: Contextual AI Prompt Construction Flowchart

Chapter 5

RESULTS AND DISCUSSION

This chapter presents the results of implementing the *Smart Boarding House Management System* by examining the actual working prototype. Each section discusses a core feature through its real user interface, describing what was built, how it works from the user's perspective, and how it connects to the technical components described in Chapter 4.

5.1 Landlord Administration Dashboard and Room Management

The landlord administration dashboard is the central control point for property managers. After logging in, landlords are directed to the room management page, which provides an immediate summary of their entire property portfolio. As shown in Figure 5.1, the top of the page displays four statistics cards that give an instant overview of the property state: the total number of rooms, rooms that are currently available for rent, rooms that are currently occupied by active tenants, and rooms under maintenance. In the example shown, the landlord manages eight rooms in total, with four rooms available, four rooms occupied, and zero rooms under maintenance. A progress bar beneath the statistics shows the occupancy rate at fifty percent.

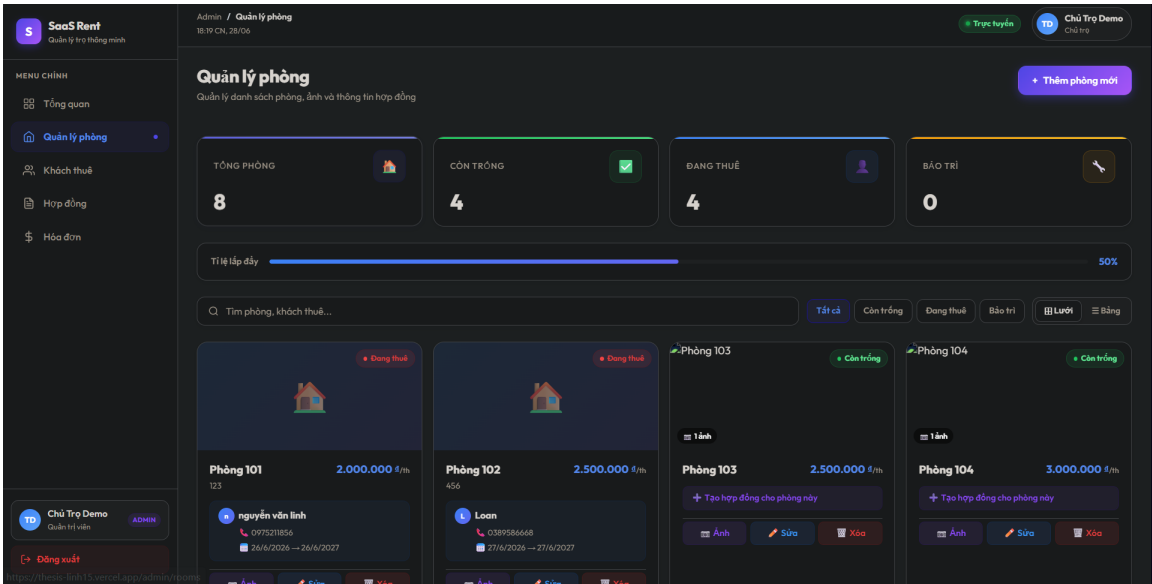


Figure 5.1: Landlord Administration Dashboard — Room Management Overview

Below the statistics, the room list is displayed as a grid of cards. Each card shows the room number, the monthly rental price, and the current room status label. The status label uses colour-coded tags so that landlords can identify available and occupied rooms at a glance without reading individual details. A filter bar above the grid allows landlords to switch between viewing all rooms, only vacant rooms, only occupied rooms, or only rooms under maintenance. This filter makes it faster to locate specific rooms, especially when managing a large number of properties.

Each room card also contains quick-action buttons so that landlords can navigate directly to that room’s detail, edit its information, or perform other operations without returning to the main menu. A prominent *Thêm phòng mới* (Add New Room) button is placed in the top-right corner of the page, allowing landlords to register a new room at any time. When a landlord creates or updates a room, the system validates all required fields on the backend before saving. Room images are uploaded to the **Cloudinary** cloud service and the secure image URLs are stored in the **PostgreSQL** database, which ensures that room photos load quickly for public visitors without consuming local server storage.

5.2 Public Room Listing and RentBot AI Chatbot

The public-facing listing page is the entry point for people searching for a room to rent. This page is accessible without any login and is designed to attract potential tenants by presenting available rooms clearly. As shown in Figure 5.2, the listing page displays a grid of room cards, each showing the room number, the monthly rental price in Vietnamese dong, and a *CÒN TRỐNG* (Available) tag that confirms the room is currently unoccupied. In the example, three rooms are visible: Phòng 103 at 2,500,000 VND, Phòng 104 at 3,000,000 VND, and Phòng 203

at 3,500,000 VND. Each card has a *Đăng ký phòng ngay* (Register for this room now) button that leads visitors to the room application form.

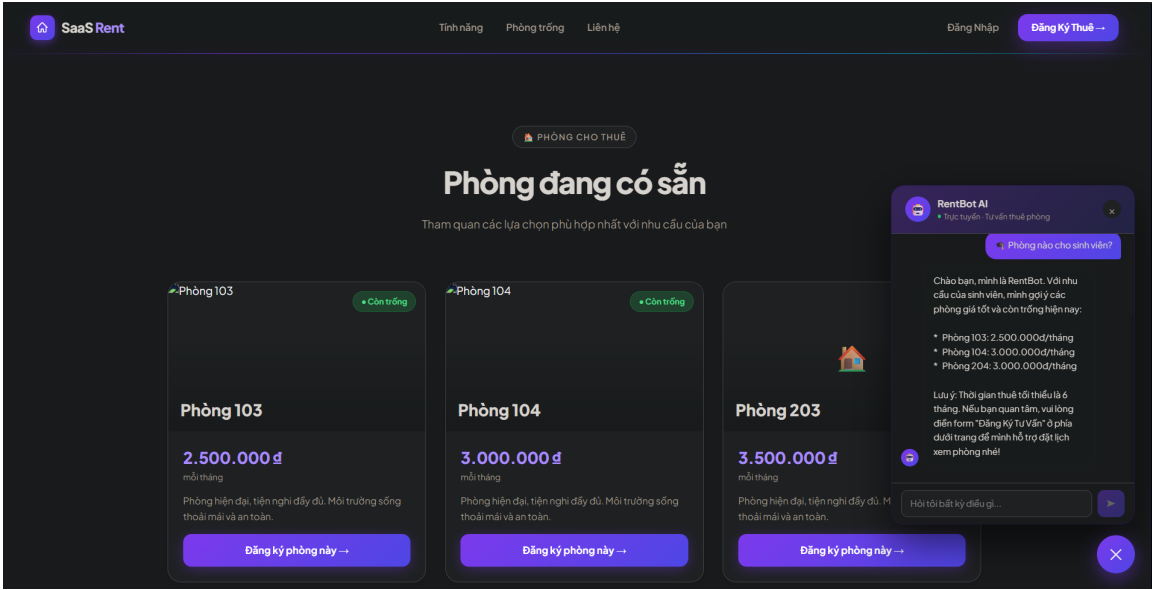


Figure 5.2: Public Room Listing Page with RentBot AI Chatbot

The most notable feature on this page is the **RentBot** chatbot widget, visible as a float-panel on the right side of the screen. RentBot is a context-aware **AI** assistant powered by the **Google Gemini** model. When a visitor opens the chatbot, the backend first queries the **PostgreSQL** database to fetch the current list of available rooms, including their room numbers, prices, floors, and descriptions. This real-time room data is then inserted into the system prompt as context before the visitor’s question is sent to the **Gemini** model. As demonstrated in the screenshot, RentBot successfully responds to the visitor by listing the available rooms with their correct prices and advising the visitor to use the registration button to apply. Because the assistant reads from live database records, it can never provide incorrect or outdated room information. This approach directly addresses the common problem of prospective tenants receiving inconsistent room availability information.

5.3 Tenant Portal and Personal AI Assistant

Once a tenant has been registered by a landlord and logs in to the system, they are directed to the tenant portal. The tenant portal is a private space where each tenant can monitor all aspects of their rental relationship. As shown in Figure 5.3, the portal greets the tenant by name and displays the current date at the top of the page, providing a personalized and welcoming experience.

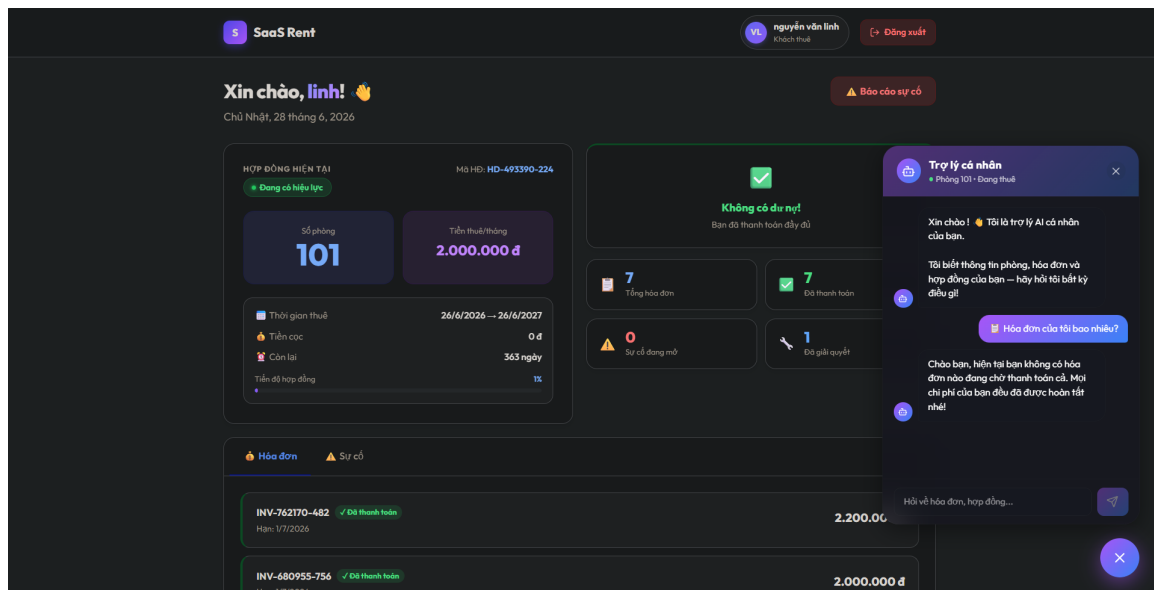


Figure 5.3: Tenant Portal Dashboard with Contract Summary and Personal AI Assistant

The left section of the portal displays the tenant's current active contract in a summary card labelled *Hợp đồng hiện tại* (Current Contract). This card shows the contract status, the room number the tenant is renting (Room 101 in this example), the rental start and end dates, and the number of days remaining on the contract. In the example shown, the tenant has a contract running from 24 June 2025 to 24 June 2027, with 363 days remaining. The tenant can also see the deposit amount and the monthly rental fee at a glance. Below the contract summary, the portal displays a list of recent invoices. Each invoice entry shows the invoice reference number, its payment status (such as *Chờ thanh toán* meaning Pending, or *Đã thanh toán* meaning Paid), and the invoice amount. This gives tenants a complete, up-to-date view of their billing history without needing to contact the landlord.

On the right side of the portal, a summary panel shows the tenant's activity counts, including the number of invoices already paid and the number of reported incidents that have been resolved. Two action buttons are placed prominently in the top-right corner: a *Báo cáo sự cố* (Report Incident) button for submitting maintenance requests, and the *Trợ lý cá nhân* (Personal Assistant) button that opens the AI chatbot.

The personal AI assistant is the most advanced feature of the tenant portal. When the tenant clicks the assistant button, the backend service queries the database to retrieve the tenant's active contract details, their full invoice history, any open incidents, and any recorded violations. All of this personal data is formatted as a structured system context and sent to the **Google Gemini** model together with the tenant's question. As demonstrated in the screenshot, the assistant can answer specific questions about the tenant's rental situation, such as informing the tenant about upcoming payment deadlines, explaining the amount still owed, or confirming whether an incident has been resolved. This level of personalization allows tenants to get precise, factual answers without calling or messaging the landlord directly.

5.4 Invoice Payment with VietQR Integration

The invoice payment workflow is one of the most practically important features of the system for both landlords and tenants. When a tenant views a pending invoice and clicks the payment button, the system opens a dedicated payment modal, as shown in Figure 5.4. This modal presents all the information a tenant needs to complete a bank transfer, without requiring them to search for the landlord's bank details manually.

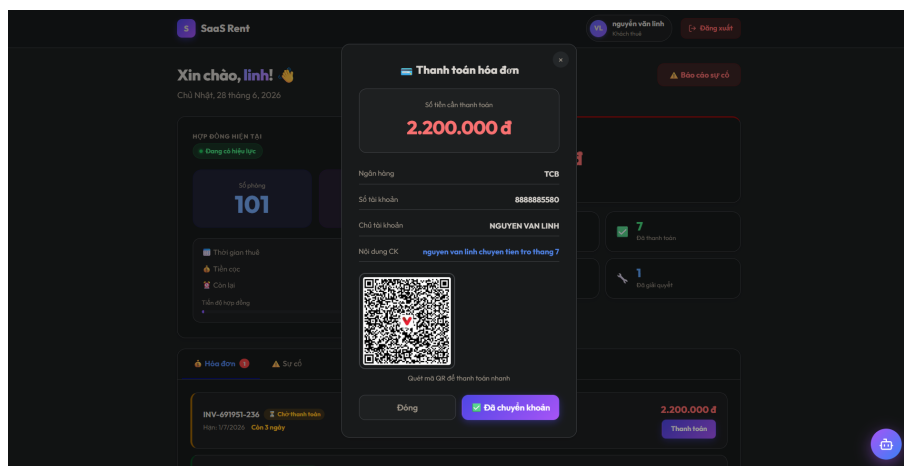


Figure 5.4: Invoice Payment Modal with VietQR Code and Bank Transfer Details

At the top of the modal, the exact amount to be paid is displayed in large text, making it immediately clear to the tenant. In the example, the tenant must pay 2,200,000 VND for the current month. Below the amount, the modal shows the complete bank transfer details: the bank name (Techcombank, abbreviated as TCB), the bank account number (8888885580), and the account holder name (NGUYEN VAN LINH). These details are retrieved from the landlord's profile stored in the database.

The most significant element of the payment modal is the automatically generated **VietQR** code displayed in the centre. This QR code is produced dynamically by the **VietQR** image API using the landlord's bank account information and the specific invoice amount. The transfer memo (Nội dung CK) is also pre-filled automatically to include the tenant's name and the invoice month, so the landlord can easily identify which payment belongs to which tenant and which month. In the example, the memo reads *nguyễn văn linh chuyển tiền trả tháng 7* (Nguyen Van Linh transfers rent for month 7). The tenant only needs to open their mobile banking application and scan this QR code to complete the full bank transfer in seconds.

After transferring the money, the tenant clicks the *Đã chuyển khoản* (Already Transferred) button at the bottom of the modal. This action sends a request to the backend, which immediately updates the invoice status to **Paid** and records the payment timestamp in the database. Simultaneously, the backend broadcasts a real-time **Socket.IO** WebSocket notification to the landlord's active dashboard session, informing them that a payment has been submitted. The

landlord can then open their bank application separately to verify that the transfer was received, and manually confirm or adjust the invoice status if necessary. Table 5.1 provides a full summary of all core features implemented in the prototype.

Feature	Status	Result
Room CRUD and image upload	Implemented	Landlords manage room records with photos stored on Cloundinary.
Public room listing	Implemented	Vacant rooms are shown on the public page for prospective tenants.
RentBot public chatbot	Implemented	AI assistant answers room availability questions using live database context.
Contract creation	Implemented	Contracts are created with atomic room status synchronization.
Monthly invoice generation	Implemented	Scheduled cron task creates unpaid invoices for all active contracts.
VietQR payment modal	Implemented	Payment modal displays bank details and a dynamically generated QR code.
Invoice payment tracking	Implemented	Tenants mark invoices paid; landlords receive real-time notifications.
Tenant portal dashboard	Implemented	Tenants view their contract summary, invoice history, and incident status.
Personal AI assistant	Implemented	AI assistant answers tenant questions using personal contract and invoice data.
Incident reporting	Implemented	Tenants submit repair requests; landlords manage and resolve them.
Real-time notifications	Implemented	Socket.IO broadcasts events to the landlord dashboard without page reload.

Table 5.1: Summary of Implemented System Features

5.5 Difficulties and Limitations

Although the prototype successfully implements all planned core features, several limitations must be acknowledged before the system can be considered ready for production deployment.

Lack of automated testing. The current codebase does not include comprehensive unit tests or integration tests. Automated testing is especially important for verifying complex business workflows such as the contract creation transaction, the monthly invoice generation cron job,

and the multi-tenant authorization filters. Without these tests, adding new features in the future carries a higher risk of accidentally breaking existing behaviour.

Manual payment verification. The invoice payment workflow requires the landlord to verify bank transfers manually by checking their bank application separately. Because the system is not connected to a live banking webhook or an open banking API, it cannot automatically detect and confirm incoming payments. The tenant must click the *Already Transferred* button to update the invoice status, and the landlord must independently confirm the transfer. This introduces a risk of human error and delays in payment confirmation.

Dependency on external AI service. All three **AI** assistant features — the notice generator, the public RentBot, and the personal tenant assistant — depend entirely on the external **Google Gemini** API. If the Google service is temporarily unavailable, rate-limited, or slow, the chatbot features will fail or respond with significant delays, directly reducing the reliability of the system for users who depend on it.

Multi-tenant security relies on application-level filtering. Data privacy between landlords is enforced by filtering every database query using the landlord identification code extracted from the **JWT** token. While this approach works correctly in the current implementation, a production-grade SaaS platform should also implement database-level row security policies and audit logging to guard against potential code-level mistakes that could expose one landlord's data to another.

No mobile application. The platform is built as a responsive web application accessible through a browser. However, tenants and landlords in Vietnam predominantly use smartphones for day-to-day activities, and a dedicated mobile application would provide a significantly smoother experience, especially for scanning QR codes and receiving push notifications.

5.6 Discussion

The results of this project demonstrate that a fully integrated web-based boarding house management platform can be built using modern open-source technologies and cloud services within the scope of a single undergraduate thesis project. By combining a modular **NestJS** backend, a server-rendered **Next.js** frontend, a **PostgreSQL** relational database, and the **Google Gemini** AI model, the system delivers a coherent set of features that directly address the real-world problems described in Chapter 1.

The most significant technical achievement is the integration of multiple system components into seamless end-to-end workflows. For example, the room status in the database changes automatically when a contract is created, expires, or is terminated. Monthly invoices are generated without any manual intervention from the landlord through the scheduled cron job. When a tenant submits payment confirmation, the landlord's dashboard is updated instantly through the **Socket.IO** WebSocket connection without requiring a page refresh. These automated workflows eliminate manual data entry, reduce the chance of human error, and save considerable time for

landlords managing multiple rooms simultaneously.

The **AI** integration proves to be one of the most practical additions to the system. The public **RentBot** reduces the number of repetitive inquiries that landlords must answer by allowing visitors to ask questions about available rooms directly on the listing page. The personal tenant assistant gives tenants a self-service channel for checking their contract terms, outstanding bills, and incident status at any time, without needing to contact the landlord. Both assistants use live data from the database rather than static scripts, which ensures that the answers are always accurate and up-to-date.

The multi-tenant architecture design ensures that the platform can serve multiple landlords from a single deployed instance without any risk of data leakage between users. Each landlord's rooms, tenants, contracts, and invoices are fully isolated through consistent use of the landlord identification code as a query filter on every database operation. This design is scalable and aligns with the SaaS business model described in the objectives of this thesis.

In conclusion, the implemented prototype successfully validates the design decisions made in Chapter 4 and fulfills the functional requirements defined in Chapter 3. The limitations identified — particularly the absence of automated tests, manual payment verification, and external AI dependency — are well-understood trade-offs that are appropriate for a prototype at this stage. Addressing these limitations through automated testing, open banking API integration, and AI fallback strategies would be the natural next steps for transforming this prototype into a production-ready product.

Chapter 6

CONCLUSION AND FUTURE WORK

This chapter summarizes the final results of this bachelor thesis and outlines the main future directions to improve the system.

6.1 Conclusion

The *Smart Boarding House Management System* successfully provides a practical foundation for digitizing rental property operations. The project solves common problems in boarding house management by centralizing room details, tenant registration, contracts, invoices, payment tracking, incidents, violations, rent requests, notifications, and AI support in a single **SaaS** platform.

From a software engineering perspective, the project demonstrates that combining modular design with clean architecture rules makes codebases highly maintainable. The backend built with **NestJS** and **Prisma ORM** ensures that database operations in **PostgreSQL** are secure and consistent. The use of custom guards like the **JwtAuthGuard** and **RolesGuard** protects private API endpoints. On the frontend, **Next.js** and **Zustand** provide a modular portal interface. The integration of supporting services like **Socket.IO** for notifications, **Cloudinary** for uploads, and **Google Gemini** for AI chat completes a modern development package. This prototype represents a secure, maintainable, and solid base for a commercial rental application.

6.2 Future Work

Although the prototype works successfully, several technical improvements can be added in future versions to make it ready for production.

6.2.1 Production Payment Integration

The current payment verification process is manual. To automate this, the system should integrate payment gateway APIs such as **PayOS**, **MoMo**, or **VNPay** and listen to secure database

webhooks. When a tenant scans the **VietQR** code and transfers money, the payment provider will automatically send a secure HTTP post request to the backend. The backend will parse the webhook, verify the invoice ID, update the invoice status to paid, and send an instant notification to both the landlord and tenant, eliminating manual verification.

6.2.2 Advanced Utility Billing

The utility billing module can be improved by tracking consumption history. Future versions should save previous and current meter readings in the database, allowing the system to calculate the difference between readings and automatically compute the electricity or water cost. The system can also run consistency checks to warn the landlord if a room has abnormally high consumption compared to previous months.

6.2.3 Stronger Multi-Tenant Security

To prepare for large-scale SaaS deployment, database-level security must be hardened. Future work should implement **Row-Level Security (RLS)** in **PostgreSQL** to enforce isolation at the database layer. Additionally, the backend should include security auditing logs to record database queries, rate limiters using the NestJS throttler module to prevent automated attacks, and strict origin checks on WebSockets to secure real-time connections.

6.2.4 Mobile Tenant Experience

Tenants primarily use mobile phones to interact with the boarding house system. Developing a native mobile application using **React Native** or **Flutter**, or converting the frontend into a **Progressive Web App (PWA)**, will make it much easier for tenants to view bills and report incidents. This will also allow the system to send mobile push notifications directly to the tenant's lock screen when a new invoice is created.

6.2.5 Analytics and Forecasting

The landlord dashboard can be expanded with analytics to track business performance. The system can calculate metrics like monthly recurring revenue, overall occupancy rate, average repair resolution times, and outstanding debts. Furthermore, machine learning models or Gemini forecasting can analyze historical data to help landlords predict vacancy risks and manage cash flow.

6.2.6 Document and Contract Management

The document management features can be expanded to include paperless contract signing. The backend can generate PDF lease documents dynamically and integrate electronic signature ser-

vices like **DocuSign** or local digital signature certificates. This will allow landlords and tenants to sign contracts digitally, making them legally binding and securely stored in the system.

REFERENCES

- [1] M. Jones, J. Bradley, and N. Sakimura. Json web token (jwt). RFC 7519, RFC Editor, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519>.
- [2] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] NestJS. Nestjs documentation, 2026. Accessed: 2026-06-27. URL: <https://docs.nestjs.com/>.
- [4] Prisma. Prisma documentation, 2026. Accessed: 2026-06-27. URL: <https://www.prisma.io/docs>.
- [5] Socket.IO. Socket.io documentation, 2026. Accessed: 2026-06-27. URL: <https://socket.io/docs/>.
- [6] Tesseract.js. Tesseract.js documentation, 2026. Accessed: 2026-06-27. URL: <https://tesseract.projectnaptha.com/>.
- [7] Vercel. Next.js documentation, 2026. Accessed: 2026-06-27. URL: <https://nextjs.org/docs>.